Module 2: Greedy Algorithms

Serge Gaspers (K17 506)

Course Admins: cs3821@unsw.edu.au
(Liam, Gerald, Song)

School of Computer Science and Engineering
UNSW Sydney

Term 3, 2025

Overview

1. Motivation
2. Optimal Selection
2.1 Activity selection
2.2 Cell tower placement
3. Detour: graph classes
4. Optimal Ordering
4.1 Tape storage
4.2 Minimising job lateness
5. Optimal Merging
5.1 The Huffman Code
6. Applications to graphs
6.1 Directed graph structure
6.2 Single source shortest paths
7. Matroids
7.1 Minimum spanning trees
8. Puzzle

Table of Contents

1. Motivation
2. Optimal Selection
2.1 Activity selection
2.2 Cell tower placement
3. Detour: graph classes
4. Optimal Ordering
4.1 Tape storage
4.2 Minimising job lateness
5. Optimal Merging
5.1 The Huffman Code
6. Applications to graphs
6.1 Directed graph structure
6.2 Single source shortest paths
7. Matroids
7.1 Minimum spanning trees
8. Puzzle

Example problem: Frozen Yoghurt I

Problem

There are n flavours of frozen yoghurt at your local shop. The ith flavour is dispensed from a machine of capacity c_i litres and contributes d_i/c_i deliciousness per litre, i.e., the entire machine's worth contributes d_i.

You have a giant tub of capacity C litres. You want to fill the tub so as to maximise the total deliciousness.

Problem

Instance: A list of n items described by their weights w_i and values v_i, and a weight limit W of your knapsack. You can take any fraction between 0 and 1 of each item.

Task: Select a non-negative quantity of each item, with total weight not exceeding W and maximum total value.

How do we choose which items to take?

- Items with <u>low weight</u> are better.
- Items with <u>high value</u> are better.

Solution

Fill the entire knapsack with the item of highest value per unit weight!

If you run out of that item, use the second best item by this ranking, and so on.

In other words, select the item with the largest ratio $\frac{v_i}{w_i}$ and pack as much of that item as you can, before trying the pack the item with the next largest ratio.

Example problem: Frozen Yoghurt II

Problem

Sadly, the frozen yoghurt shop has now closed down, so you must go to the supermarket instead. There, you can find n flavours of frozen yoghurt in tubs. The ith flavour comes in a tub of capacity c_i litres and contributes d_i deliciousness.

You again want to buy C litres, with maximum total deliciousness.

0-1 Knapsack

Problem

Instance: A list of n discrete items described by their weights w_i and values v_i, and a maximal weight limit W of your knapsack.

Task: Find a subset S of the items with total weight not exceeding W and maximal total value.

- As before, we want large value and low weight items.
- But this time we can't take fractions of an item, and we can't take multiple copies of an item. How does that change things?

0-1 Knapsack

- Can we always choose the item of highest value per unit weight?
- Assume there are just three items with weights and values:

$$A(1\,\text{kg}, \$60), B(2\,\text{kg}, \$100), C(3\,\text{kg}, \$120)$$

  and a knapsack of capacity $W = 5\,\text{kg}$.
- The greedy strategy would choose items A and B, while the optimal solution is to take items B and C!
- So when do greedy algorithms work?
- Unfortunately there is no easy rule ...

Question

What is a greedy algorithm?

Answer

A greedy algorithm is one that solves a problem by dividing it into stages, and rather than exhaustively searching all the ways to get from one stage to the next, instead only considers the choice that appears best.

This obviously reduces the search space, but it is not always clear whether the locally optimal choice leads to the globally optimal outcome.

Question

Are greedy algorithms always correct?

Answer

No!

Suppose you are searching for the highest point in a mountain range. If you always climb upwards from the current point in the steepest possible direction, you will find a peak, but not necessarily the highest point overall.

Question

Is there a framework to decide whether a problem can be solved using a greedy algorithm?

Answer

Yes: matroids (for a narrow definition of greedy) and matroid embeddings (for a more inclusive definition of greedy). It requires finding/defining the right combinatorial structure, which is not necessarily easier than formulating a greedy algorithm directly.

With experience, you will develop an intuition for whether greedy algorithms are useful for a particular problem.

- Many greedy algorithms are relatively simple to describe.
- The trade-off is that they by definition don't consider all of the possibilities.
- This leaves some work to do in analysing correctness.
- Why did you not need to consider those other possibilities? Equivalently, why were the possibilities you considered no worse than the others?

Question

How do we prove that a greedy algorithm is correct?

Answer

There are two main methods of proof:

1. Greedy stays ahead: proves that the greedy solution is optimal by showing that, at every iteration, no other sequence of choices could do better than the greedy algorithm.
2. Exchange argument: proves that the greedy solution is optimal by gradually transforming any optimal solution into the solution computed by the greedy algorithm, whilst maintaining optimality.

Table of Contents

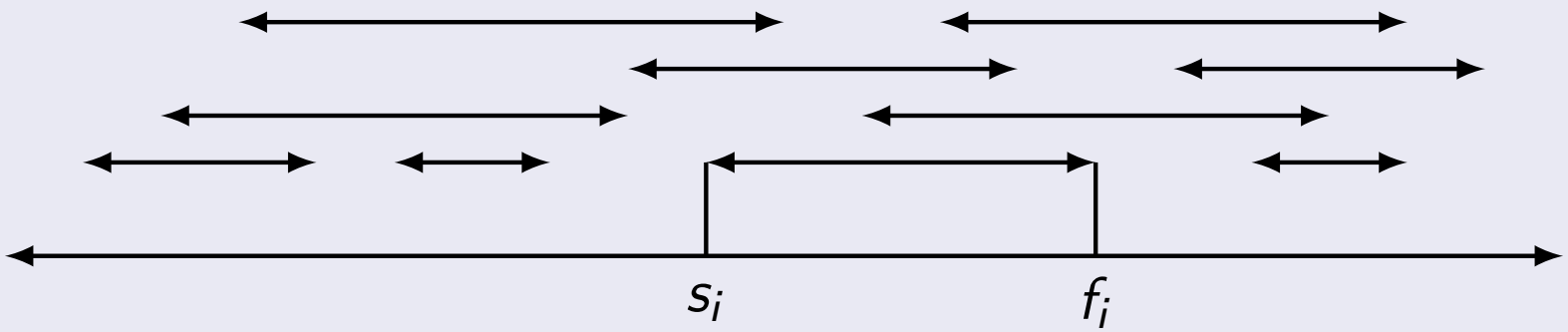
1. Motivation
2. Optimal Selection
2.1 Activity selection
2.2 Cell tower placement
3. Detour: graph classes
4. Optimal Ordering
4.1 Tape storage
4.2 Minimising job lateness
5. Optimal Merging
5.1 The Huffman Code
6. Applications to graphs
6.1 Directed graph structure
6.2 Single source shortest paths
7. Matroids
7.1 Minimum spanning trees
8. Puzzle

Table of Contents

1. Motivation
2. Optimal Selection
2.1 Activity selection
2.2 Cell tower placement
3. Detour: graph classes
4. Optimal Ordering
4.1 Tape storage
4.2 Minimising job lateness
5. Optimal Merging
5.1 The Huffman Code
6. Applications to graphs
6.1 Directed graph structure
6.2 Single source shortest paths
7. Matroids
7.1 Minimum spanning trees
8. Puzzle

Problem

Instance: A list of n activities, with starting times s_i and finishing times f_i.



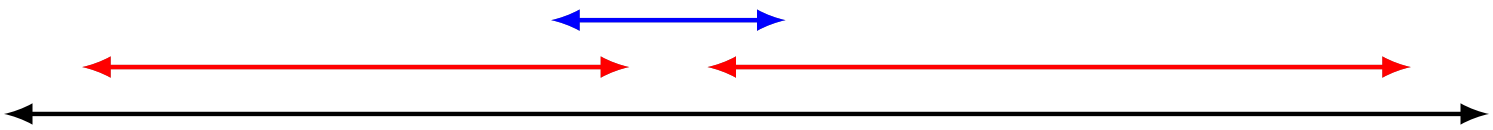
Task: Select a maximum number of pairwise compatible activities. Two activities are compatible if they do not overlap.

Possible ideas:

- Choose shorter activities first?
- Choose activities which have fewest conflicts?
- Choose activities with earlier end times first?

Attempt 1

Always choose the shortest activity which does not conflict with the previously chosen activities, then remove the conflicting activities and repeat.



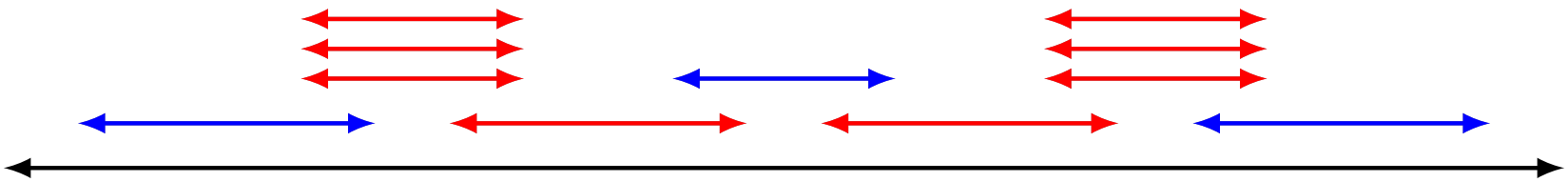
In the above example, our proposed algorithm chooses the shortest activity (blue), then has to discard both the red activities, which is worse than picking both of the longer activities.

Possible ideas:

- ~~Choose shorter activities first?~~
- Choose activities which have fewest conflicts?
- Choose activities with earlier end times first?

Attempt 2

Maybe we should always choose an activity which conflicts with the fewest possible number of the remaining activities? It may appear that in this way we minimally restrict our next choice . . .



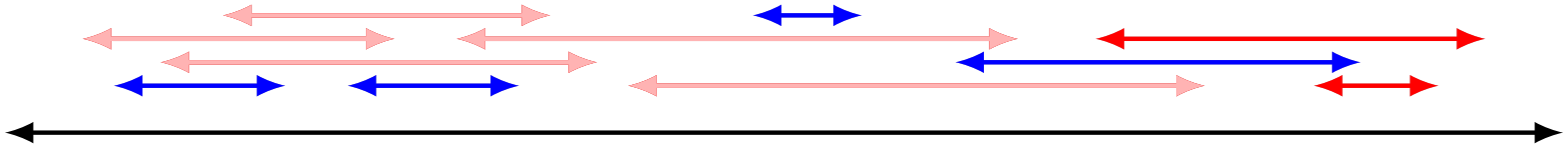
As appealing this idea is, the above figure shows this again does not work!

Possible ideas:

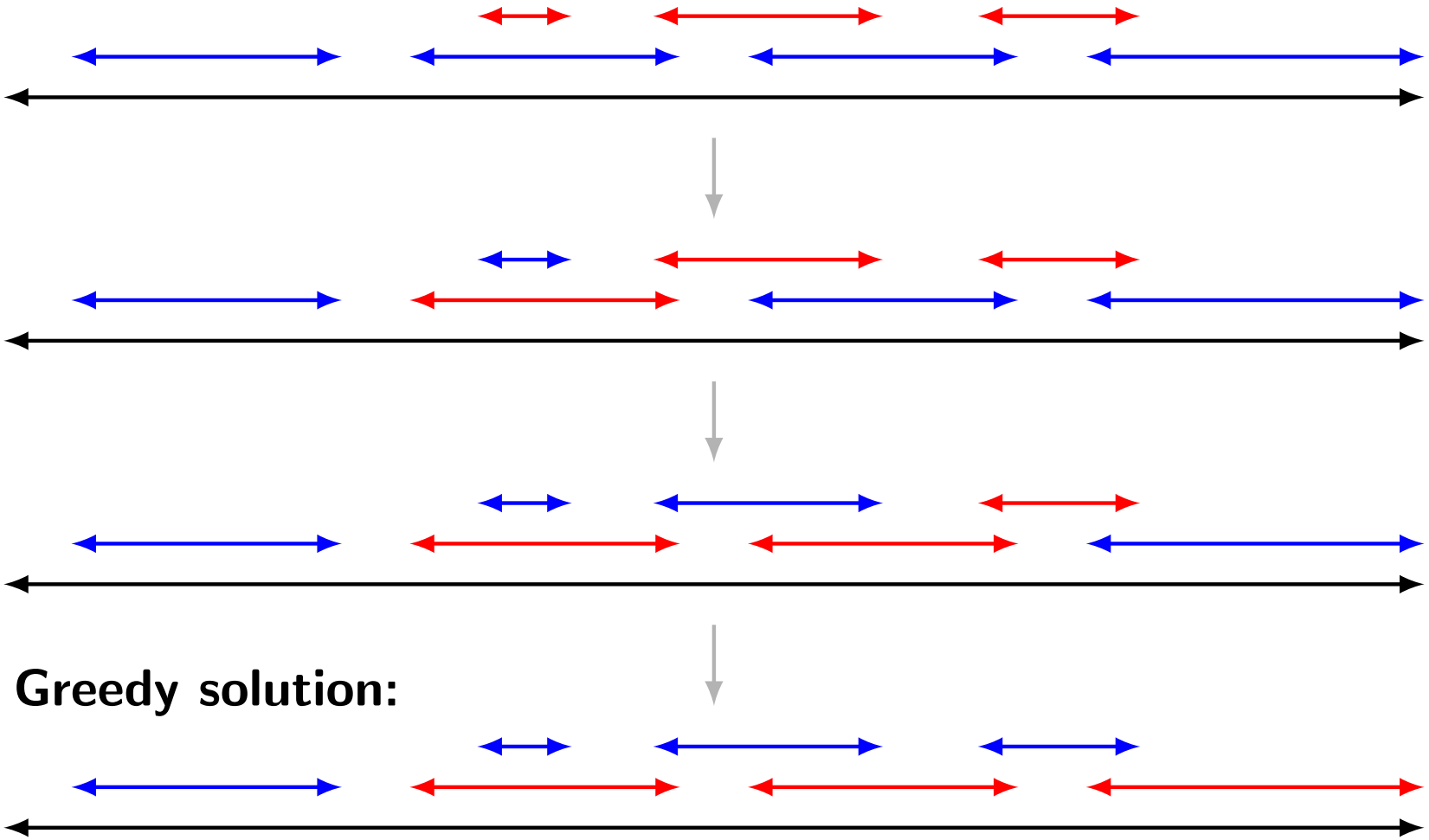
- ~~Choose shorter activities first?~~
- ~~Choose activities which have fewest conflicts?~~
- Choose activities with earlier end times first?

Attempt 3 (Solution)

Among those activities which do not conflict with the previously chosen activities, always choose the activity with the earliest end time (breaking ties arbitrarily).



Optimal solution:



Activity Selection: correctness

To prove the correctness of the algorithm, we will use an <u>exchange argument</u>.

- Let $G = \{g_1, \ldots, g_r\}$ denote the solution produced by the greedy algorithm and let $O = \{o_1, \ldots, o_s\}$ be an optimal solution. We assume that $f_{g_1} \leq \cdots \leq f_{g_r}$ and $f_{o_1} \leq \cdots \leq f_{o_s}$.
- If $G \not\subseteq O$, then let k denote the first index such that $g_k \neq o_k$.
- By the greedy choice, $f_{g_k} \leq f_{o_\ell}$ for all $\ell \geq k$, and $g_k \notin O$.
- Let $O' := (O \setminus \{o_k\}) \cup \{g_k\}$. We have that $|O'| = |O|$. We now argue that O' is a feasible solution.

Claim

The set $O' := (O \setminus \{o_k\}) \cup \{g_k\}$ is a set of compatible activities.

Proof.

By construction, g_k does not conflict with any activity in the set $\{o_1, \ldots, o_{k-1}\}$ since $o_i = g_i$ for each $1 \leq i \leq k-1$. Moreover, since g_k is the activity with the earliest ending time among the activities that do not conflict with $\{o_1, \ldots, o_{k-1}\}$, we have that

$$f_{g_k} \leq f_{o_k} \leq s_{o_i},$$

for any $i \in \{k+1, \ldots, s\}$. Therefore, g_k does not conflict with any activity in the set $\{o_{k+1}, \ldots, o_s\}$, which implies that O' is a feasible solution. $\square$

Activity Selection: correctness

- We conclude that, by induction, there is an optimal solution O that contains all of G.
- It remains to consider the case where $G \subseteq O$. If $G = O$, then the greedy solution is optimal. We now argue that $|O| \not> |G|$.
- If there were activities $o_{r+1}, \ldots, o_s$ that do not conflict with activities $g_1, \ldots, g_r$, then the greedy algorithm would choose additional activities. We conclude that the greedy solution is optimal.

Activity Selection: Time Complexity

What is the time complexity of the algorithm?

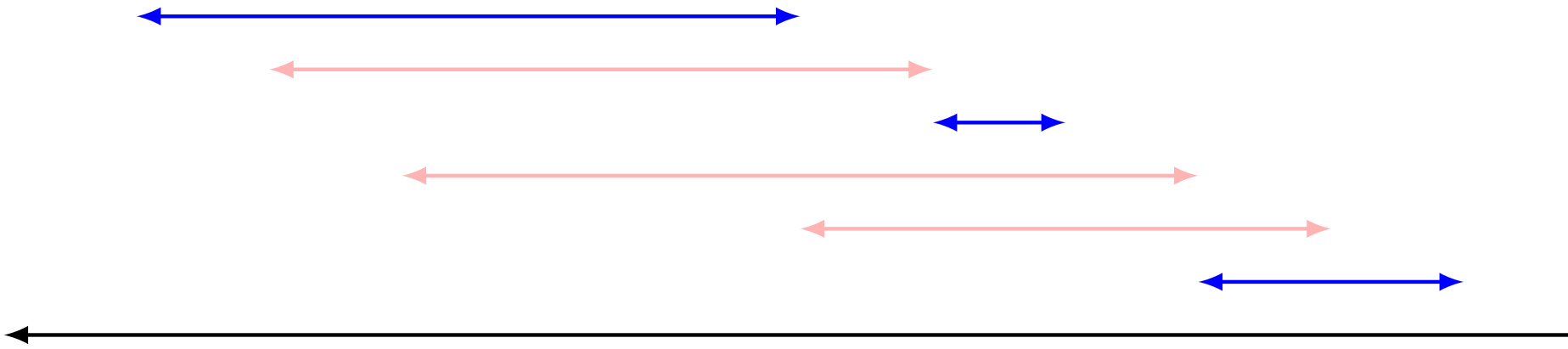
- We represent activities by ordered pairs of their starting and their finishing times and sort them in increasing order of their finishing time (the second coordinate). Using mergesort, this takes $O(n \log n)$ time.

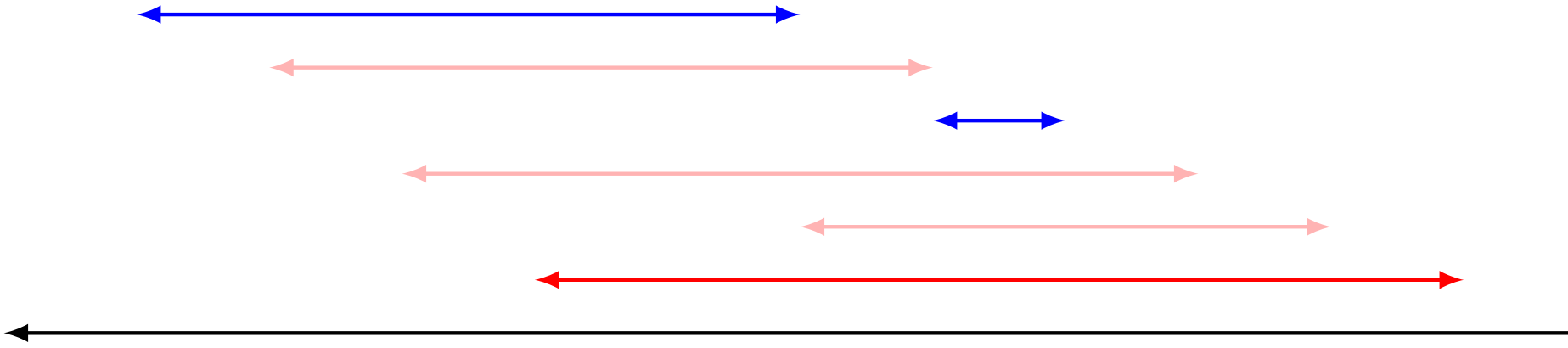
- We go through this sorted list in order. How do we tell whether an activity conflicts with the already chosen activities?

Activity Selection: Time Complexity

- Suppose we need to decide whether to add activity i to the solution, and all activities with earlier finishing times have already been processed.
- If all previously chosen activities finished before s_i, activity i can be added without a conflict.
- Otherwise, there will be a clash, so we discard activity i.
- We would prefer not to go through all previously added activities each time.

Activity Selection: Time Complexity





Activity Selection: Time Complexity

- We need only keep track of the latest finishing time among added activities.
- Since we process activities in increasing order of finishing time, this is just the finishing time of the last added activity.
- Every activity is therefore either added (and the last finishing time updated) or discarded in constant time, so this part of the algorithm takes $O(n)$ time.
- So our algorithm has two steps:
  1. **Sorting:** $O(n \log n)$ time.
  2. **Construct solution:** $O(n)$ time
- Thus, the running time of the algorithm is

$$O(n \log n) + O(n) = O(n \log n).$$

A related problem

Instance: A list of n activities with starting times s_i and finishing times $f_i = s_i + d$; thus, all activities are of the same duration.

Task: Select a set of compatible (i.e., non-overlapping) activities of <u>maximum total duration</u>.

Solution

Since all activities are of the same duration, this is equivalent to finding a selection with a largest number of compatible activities, i.e., the previous problem.

Question

What happens if the activities are not all of the same duration?

Solution

The greedy strategy no longer works. Our earlier exchange argument relied on any interval being just as good as any other for the objective, and this is not the case anymore.

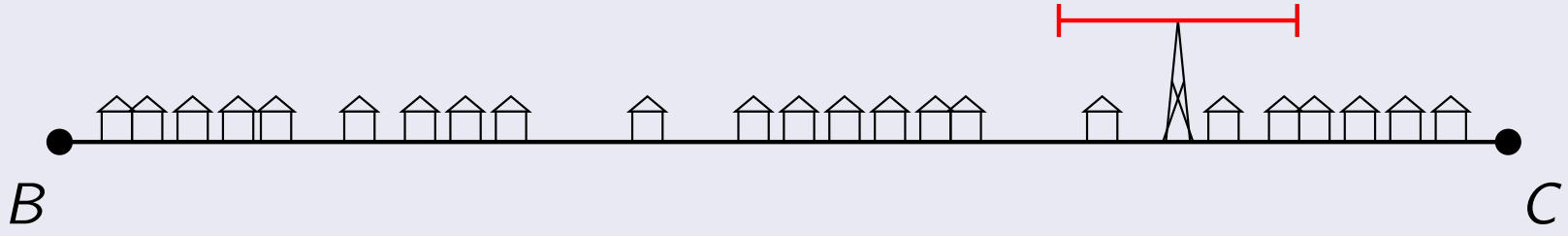
We can't prune the search space as aggressively in this variant, so we'll need an "efficient brute force" . . . dynamic programming.

Table of Contents

1. Motivation
2. Optimal Selection
2.1 Activity selection
2.2 Cell tower placement
3. Detour: graph classes
4. Optimal Ordering
4.1 Tape storage
4.2 Minimising job lateness
5. Optimal Merging
5.1 The Huffman Code
6. Applications to graphs
6.1 Directed graph structure
6.2 Single source shortest paths
7. Matroids
7.1 Minimum spanning trees
8. Puzzle

Problem

Instance: Along the long, straight road from Balladonia (in the West) to Caiguna (in the East), houses are scattered quite sparsely, sometimes with long gaps between two consecutive houses. Telstra must provide mobile phone service to people who live alongside the road, and the range of Telstra's cell tower is 5km.



Task: Place a smallest number of cell towers alongside the road, that is sufficient to cover all houses.

Cell Towers

- Let's attempt a greedy algorithm, processing the houses west to east.
- The first house must be covered by some tower, which we place 5km to the east of this house.
- This tower may cover some other houses, and we consider the first house that is out of range of this tower. We then place a second tower 5km to the east of that house.
- Continue in this way until all houses are covered.

- At each house, we need to decide whether to place a new tower. This can be done in constant time by referring to the most recently created tower, which can itself be updated in constant time if necessary.

- Therefore this algorithm runs in $O(n)$ time if the houses are provided in order, and $O(n \log n)$ time otherwise.

Proof idea.

- For variety, we will present an alternative method of proof called greedy stays ahead.
- The key claim is: for all k, the kth tower in the greedy solution is at least as far east as the kth tower in any other placement.
- If this is true, then it would be impossible to cover all the houses using fewer towers.
- To prove the key claim, we use induction.

Claim

For all k, the kth tower in the greedy solution is at least as far east as the kth tower in any other solution.

Base case:

- The base case is $k = 1$.
  - Greedy places the first tower 5km east of the first house.
  - If an alternative solution A had the first tower further east, it would not cover this first house, and A would not be a valid solution.
  - So the claim is true for $k = 1$.
- We next need to prove the inductive step...

Claim

For all k, the kth tower in the greedy solution is at least as far east as the kth tower in any other solution.

Inductive step:

- Suppose the claim is true for $k-1$. So $a_{k-1} \leq g_{k-1}$.
- Consider tower k.
  - Greedy placed towers at g_{k-1} and g_k. So there is a house h with
    $$g_{k-1} + 5 < h = g_k - 5.$$
  - The alternative solution placed towers at a_{k-1} and a_k. <u>Assume</u> $a_k > g_k$. Then
    $$a_{k-1} + 5 \leq g_{k-1} + 5 < h = g_k - 5 < a_k - 5.$$
  - So house h is not covered by tower a_{k-1} or tower a_k. This contradicts that A is a valid solution.
  - So our assumption was false: we must have $a_k \leq g_k$.

We have proved the claim:

Claim

For all k, the kth tower in the greedy solution is at least as far east as the kth tower in any other solution.

We now use this claim to prove that no other solution uses fewer towers than greedy.

Proof that greedy is optimal:

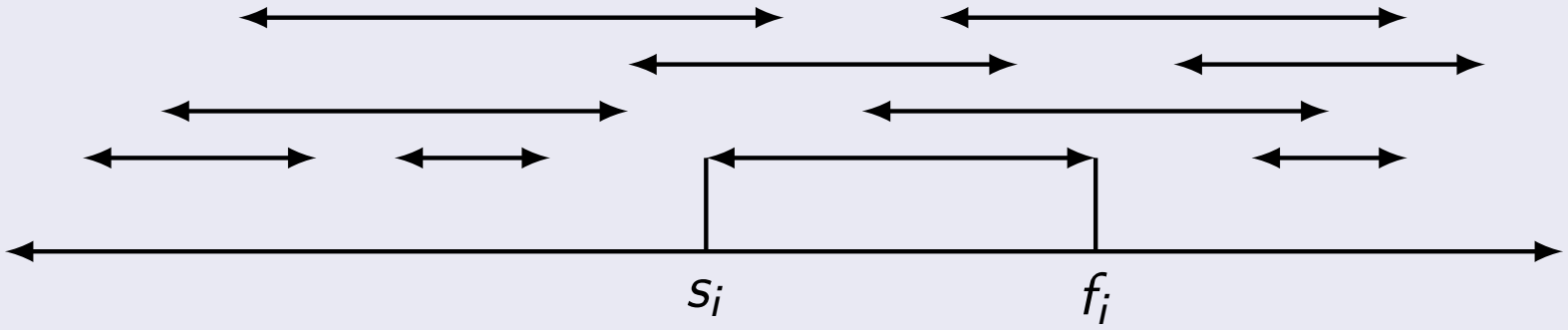
- Suppose greedy uses r towers, placed at $g_1, \ldots, g_r$.
- There is a house h at $g_r - 5$, which isn't covered by the previous tower. Therefore $h > g_{r-1} + 5$.
- From the induction above, we know that for any other placement of $r - 1$ towers $a_1, \ldots, a_{r-1}$, we have $a_{r-1} \leq g_{r-1}$.
- Therefore house h can't be covered in the first $r - 1$ towers in <u>any</u> valid solution.
- So at least r towers are needed. Therefore greedy is optimal.

Table of Contents

1. Motivation
2. Optimal Selection
2.1 Activity selection
2.2 Cell tower placement
3. Detour: graph classes
4. Optimal Ordering
4.1 Tape storage
4.2 Minimising job lateness
5. Optimal Merging
5.1 The Huffman Code
6. Applications to graphs
6.1 Directed graph structure
6.2 Single source shortest paths
7. Matroids
7.1 Minimum spanning trees
8. Puzzle

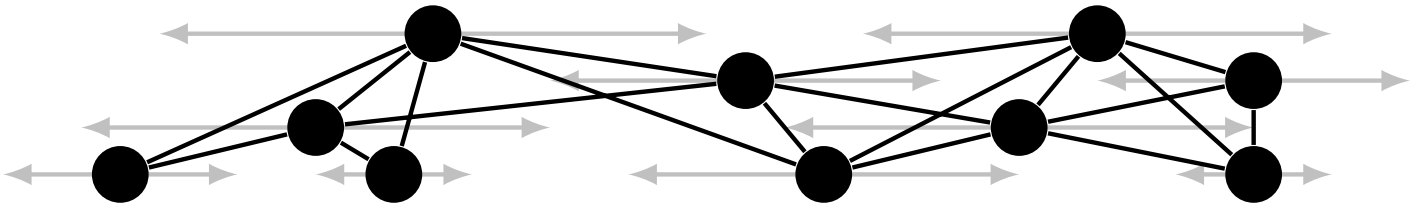
Problem

Instance: n intervals



Task: Find a largest subset of non-overlapping intervals.

The intersection graph of the intervals has a vertex for each interval and an edge between every pair of overlapping intervals.



Definition

A graph $G = (V, E)$ is an interval graph if it is possible to associate an interval of the real line with each vertex such that G is the intersection graph of these intervals.

- A clique is a subset $C \subseteq V$ such that each pair of vertices in C is adjacent.
- An independent set is a subset $I \subseteq V$ such that no pair of vertices in I is adjacent.
- A clique/independent set is (inclusion-wise) maximal if it is not contained in any larger clique/independent set.

Exercise

Prove the following statement. If a graph $G = (V, E)$ is an interval graph, then G has $O(|V|)$ maximal cliques, and they can be enumerated in $O(|V| + |E|)$ time.

Recognition: Given a graph G, is G an interval graph?

This can be solved in $O(n+m)$ time, where n is the number of vertices and m is the number of edges of G. The algorithm also returns an interval representation.

3-Colourability [?]	Linear
Clique [?]	Polynomial
Clique cover [?]	Linear
Colourability [?]	Linear
Domination [?]	Linear
Feedback vertex set [?]	Linear
Graph isomorphism [?]	Linear
Hamiltonian cycle [?]	Linear
Hamiltonian path [?]	Polynomial
Independent dominating set [?]	Linear
Independent set [?]	Linear
Maximum cut [?]	NP-complete

Source: www.graphclasses.org

Activity selection revisited

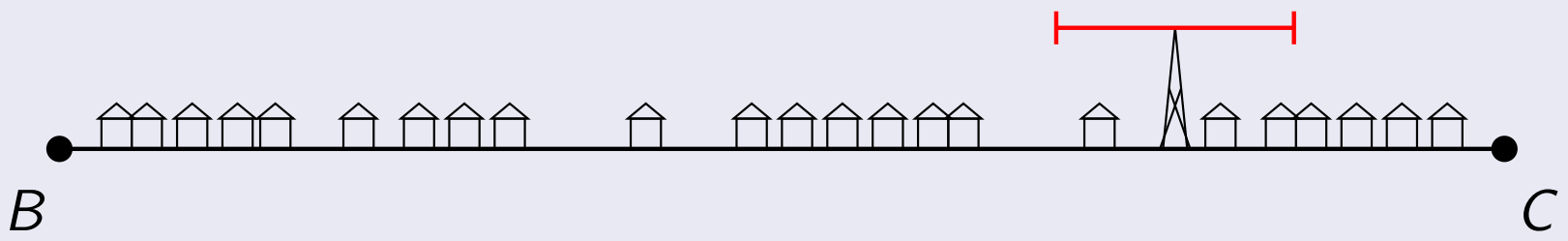
- A set of intervals is non-overlapping iff the corresponding vertices form an independent set in the intersection graph.

- Activity selection is equivalent to Independent Set on interval graphs (assuming sorted intervals are provided along with the input)

An abstraction of cell tower placement

Problem

Instance: Houses placed on a line
Task: Placing a smallest number of cell towers with 5km range alongside the road, such that all houses are covered.



Equivalent problem: Stabbing unit intervals

Instance: n intervals of length 1
Task: Find a smallest subset of points that intersects every interval.

Unit interval graphs

Definition

A graph $G = (V, E)$ is a unit interval graph (a.k.a., proper interval graph) if it is possible to associate an interval the real line of length 1 with each vertex such that G is the intersection graph of these intervals.

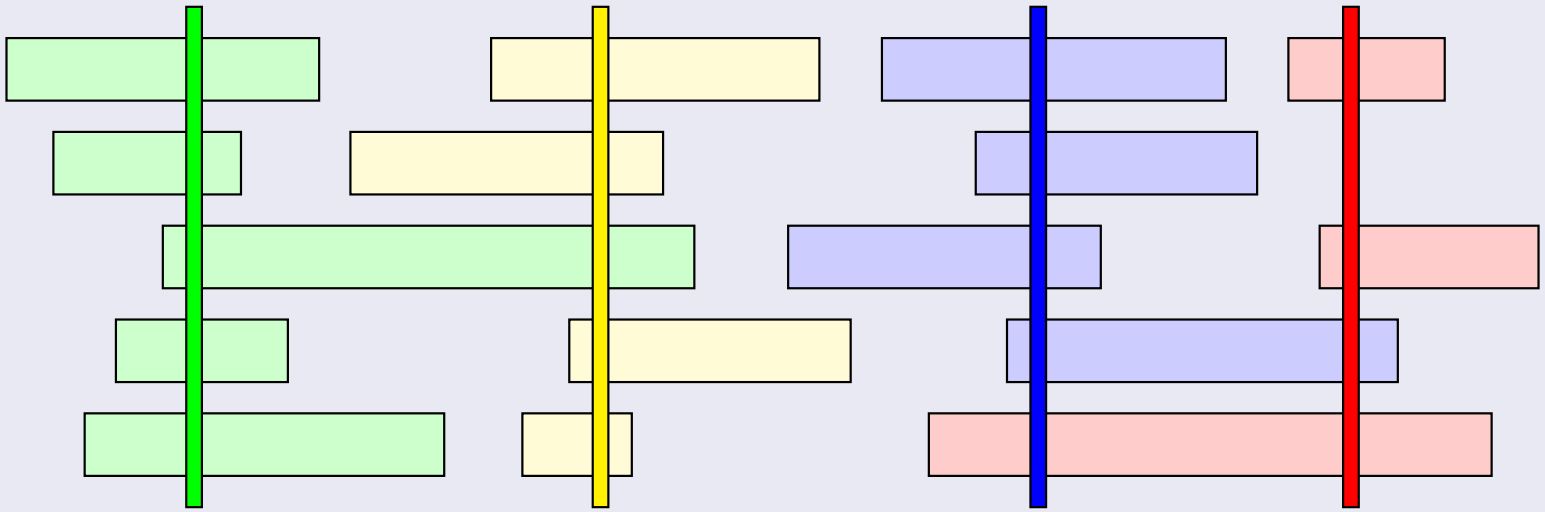
- Recognition can be solved in linear time
- Some problems that are hard on interval graphs become polynomial-time solvable on unit interval graphs, e.g., Maximum Cut

Cell tower problem revisited

- The following problems are equivalent:
  - Placing a smallest number of cell towers with 5km range alongside the road, such that all houses are covered
  - Stabbing a set of unit intervals with a smallest number of points
  - Finding a smallest clique cover in a unit interval graph (where the input also contains a unit interval representation of the graph)
- A clique cover of a graph $G = (V, E)$ is a partition of V into cliques. The size of the clique cover is the number of cliques in the partition.
- Can we generalize the greedy algorithm for cell tower placement to solve Clique Cover in interval graphs?

Problem

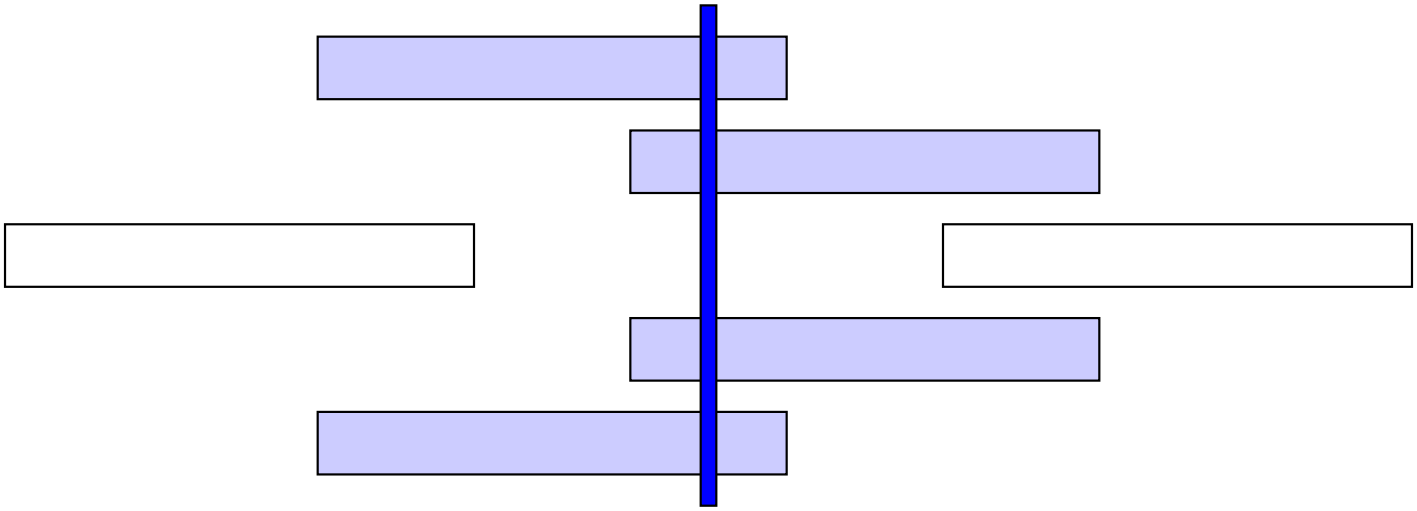
Instance: Let X be a set of n intervals on the real line, described by two arrays $X_L[1..n]$ and $X_R[1..n]$, representing their left and right endpoints. We say that a set P of points stabs X if every interval in X contains at least one point in P.



Task: Describe and analyse an efficient algorithm to compute the smallest set of points that stabs X.

Attempt 1

Is it a good idea to stab the largest possible number of intervals?



No! In the above example, this strategy needs three stabbing points rather than two.

Exercise

Design and analyse a greedy algorithm for the interval stabbing problem.

Hint

The interval which ends the earliest has to be stabbed somewhere.

What is the best place to stab it?

Intersection graphs

Various graph classes are intersection graphs of objects in a space.

- Interval graphs . . . intervals on the real line
- Unit interval graphs . . . unit-length intervals on the real line
- Circular-arc graphs . . . arcs on a circle
- Circle graphs . . . chords of a circle
- Chordal graphs . . . subtrees of a tree
- Unit disk graphs . . . unit disks in the plane
- Permutation graphs . . . line segments between two parallel lines
- Line graphs . . . edges of a graph
- . . .

- Information System on Graph Classes and their Inclusions (ISGCI) www.graphclasses.org
- SageMath has an interface to ISGCI
- Algorithmic Graph Theory and Perfect Graphs, Martin C. Golumbic, 2004
- Graph Classes: A Survey, Andreas Brandstädt, Van Bang Le, Jeremy P. Spinrad, 1999

Definition

The chomatic number of a graph is the minimum number of colors needed to color the vertices such that no two adjacent vertices share the same color.
A graph is perfect if for every induced subgraph, the size of the largest clique equals the chromatic number.

Note

For two graphs G, H, we say that H is an induced subgraph of G if H can be obtained from G by deleting vertices (and the edges incident to the deleted vertices).
H is a subgraph of G if H can be obtained from G by deleting vertices and edges.

Theorem (Perfect Graph Theorem, Lovász 1972)

A graph is perfect iff its complement is perfect.

Therefore, a graph is perfect iff for every induced subgraph, the size of the largest independent set equals the clique cover number.

Theorem (Strong Perfect Graph Theorem, Chudnovsky, Robertson, Seymour, Thomas 2006)

A graph is perfect iff it contains no odd hole or odd anti-hole.

A hole is an induced cycle of length at least 4. An anti-hole is the complement of a hole.

Exercise

Show that interval graphs are perfect.

Interpolating between graph classes and general graphs 55

- There are fast algorithms for interval graphs
- But what to do if our input graph is not in the class?
  - consider graph classes that include interval graphs, e.g., circular-arc graphs, chordal graphs, perfect graphs
  - k-interval graphs where every vertex corresponds to a set of at most k intervals
  - k-gap interval graphs where the total number of intervals used is $n + k$ for n vertices
  - find a (small) set of vertices whose removal puts the graph in the class

Table of Contents

1. Motivation
2. Optimal Selection
2.1 Activity selection
2.2 Cell tower placement
3. Detour: graph classes
4. Optimal Ordering
4.1 Tape storage
4.2 Minimising job lateness
5. Optimal Merging
5.1 The Huffman Code
6. Applications to graphs
6.1 Directed graph structure
6.2 Single source shortest paths
7. Matroids
7.1 Minimum spanning trees
8. Puzzle

Table of Contents

1. Motivation
2. Optimal Selection
2.1 Activity selection
2.2 Cell tower placement
3. Detour: graph classes
4. Optimal Ordering
4.1 Tape storage
4.2 Minimising job lateness
5. Optimal Merging
5.1 The Huffman Code
6. Applications to graphs
6.1 Directed graph structure
6.2 Single source shortest paths
7. Matroids
7.1 Minimum spanning trees
8. Puzzle

Problem

Instance: A list of n files of lengths L_i which need to be stored on a tape.

Each file is equally likely to be needed. To retrieve a file, one must start from the beginning of the tape and scan it until the file is found and read.

Task: Order the files on the tape so that the average (expected) retrieval time is minimised.

Solution

Store files in order of increasing length.

- Suppose we store the files in order $L_1, L_2, \ldots, L_n$
- The probability we want to read any one of the files is $\frac{1}{n}$.
- So the expected retrieval time is

$$
\begin{aligned}
E &= \sum_{i=1}^{n} \frac{1}{n}(L_1 + \cdots + L_i) = \frac{1}{n}\sum_{i=1}^{n}(L_1 + \cdots + L_i) \\
&= \frac{1}{n}\Big(L_1 + (L_1 + L_2) + (L_1 + L_2 + L_3) + \cdots + (L_1 + \ldots + L_n)\Big) \\
&= \frac{1}{n}\Big(nL_1 + (n-1)L_2 + (n-2)L_3 + \cdots + 2L_{n-1} + L_n\Big).
\end{aligned}
$$

- **Claim.** This is minimised if $L_1 \leq L_2 \leq L_3 \leq \ldots \leq L_n$.
- So we sort the files by increasing length for an $O(n \log n)$ solution.

Claim

The "scaled" retrieval time

$$nL_1 + (n-1)L_2 + (n-2)L_3 + \ldots + 2L_{n-1} + L_n,$$

is minimised if $L_1 \leq L_2 \leq \ldots \leq L_n$.

We prove this by contradiction, which is probably the most common method for proofs of optimal ordering.

Proof (by contradiction):

- Suppose the files are <u>not</u> in ascending order by length. Then there are two consecutive files with $L_i > L_{i+1}$.
- Swapping them changes the scaled retrieval time from

$$\begin{aligned} T &= \cdots + (n-i+1)L_i + (n-i)L_{i+1} + \ldots \\ \text{to} \quad T' &= \cdots + (n-i+1)L_{i+1} + (n-i)L_i + \ldots. \end{aligned}$$

- This is a reduction because $T - T' = L_i - L_{i+1} > 0$.

- Any arrangement which isn't sorted by length can be improved upon, so the best arrangement is the one where files are ordered from shortest to longest.
- Indeed, resolving these "adjacent inversions" one-by-one would gradually return us to the sorted sequence, (our proof uses the same idea as bubble sort!)

Algorithm

Sort the files by increasing order of length using mergesort.

What if the files are not equally likely to be picked?

Problem

Instance: A list of n files of lengths L_i and probabilities to be needed p_i, $\sum_{i=1}^{n} p_i = 1$, which need to be stored on a tape.

To retrieve a file, one must start from the beginning of the tape and scan it until the file is found and read.

Task: Order the files on the tape so that the **expected** retrieval time is minimised.

We have two criteria we could order by: length L_i and probability p_i.

- Files with <u>small</u> length L_i should appear near the start
- Files with <u>large</u> probability p_i should appear near the start.

How can we sort by both of these properties at the same time?

Solution

Store files in decreasing value of the ratio $\frac{p_i}{L_i}$.

- If the files are stored in order $L_1, L_2, \ldots L_n$, then the expected retrieval time is

$$\begin{aligned} E = & L_1 p_1 + (L_1 + L_2) p_2 + (L_1 + L_2 + L_3) p_3 \\ & + \cdots + (L_1 + L_2 + L_3 + \cdots + L_n) p_n \end{aligned}$$

Claim

The expected retrieval time E is minimised if the files are ordered by decreasing p_i / L_i.

- For the sake of contradiction, suppose E is minimised, but there are two adjacent files k and $k+1$ such that $\frac{p_k}{L_k} < \frac{p_{k+1}}{L_{k+1}}$.

Tape Storage II: Proof of optimality

- What happens if we swap the order of these two files?
- Before the swap, the expected retrieval time is:

$$
\begin{aligned}
E = {} & L_1 p_1 + (L_1 + L_2) p_2 + (L_1 + L_2 + L_3) p_3 + \ldots \\
& + (L_1 + L_2 + L_3 + \ldots + L_{k-1} + L_k) p_k \\
& + (L_1 + L_2 + L_3 + \ldots + L_{k-1} + L_k + L_{k+1}) p_{k+1} \\
& + \ldots + (L_1 + L_2 + L_3 + \ldots + L_n) p_n
\end{aligned}
$$

- After the swap, the new expected retrieval time is:

$$
\begin{aligned}
E' = {} & L_1 p_1 + (L_1 + L_2) p_2 + (L_1 + L_2 + L_3) p_3 + \ldots \\
& + (L_1 + L_2 + L_3 + \ldots + L_{k-1} + L_{k+1}) p_{k+1} \\
& + (L_1 + L_2 + L_3 + \ldots + L_{k-1} + L_{k+1} + L_k) p_k \\
& + \ldots + (L_1 + L_2 + L_3 + \ldots + L_n) p_n.
\end{aligned}
$$

- Now, $E - E' = L_k p_{k+1} - L_{k+1} p_k$.

Tape Storage II: Proof of optimality

- Since $p_k/L_k < p_{k+1}/L_{k+1}$, we have that $L_{k+1}p_k < L_k p_{k+1}$.
- Thus, $E - E' = L_k p_{k+1} - L_{k+1} p_k > 0$.
- Therefore, $E > E'$. This is a contradiction, since E does not have the smallest possible retrieval time.
- Therefore, there is no <u>inversion</u> (file $k+1$ with a larger ratio p_{k+1}/L_{k+1} occurs after file k with a smaller ratio p_k/L_k).
- So the optimal sequence is the one with no inversions, i.e., in descending order of the ratio p_i/L_i.

Table of Contents

1. Motivation
2. Optimal Selection
2.1 Activity selection
2.2 Cell tower placement
3. Detour: graph classes
4. Optimal Ordering
4.1 Tape storage
4.2 Minimising job lateness
5. Optimal Merging
5.1 The Huffman Code
6. Applications to graphs
6.1 Directed graph structure
6.2 Single source shortest paths
7. Matroids
7.1 Minimum spanning trees
8. Puzzle

Problem

Instance: A list of n jobs, with durations $\texttt{duration}_i$ and deadlines $\texttt{due}_i$.
All jobs have to be completed, but only one job can be performed at any time.
If a job i is completed at a finishing time $\texttt{finish}_i > \texttt{due}_i$ then we say that it has incurred lateness $\texttt{late}_i = \texttt{finish}_i - \texttt{due}_i$.

Task: Schedule all the jobs to minimise the largest lateness.

We have two criteria we can order by, duration and deadlines.
Which is more important to find the optimal solution?

- Perform quicker tasks first? (small $\texttt{duration}_i$)
- Prioritise tasks with earlier deadlines? (small $\texttt{due}_i$)
- Maybe we need to consider both factors?

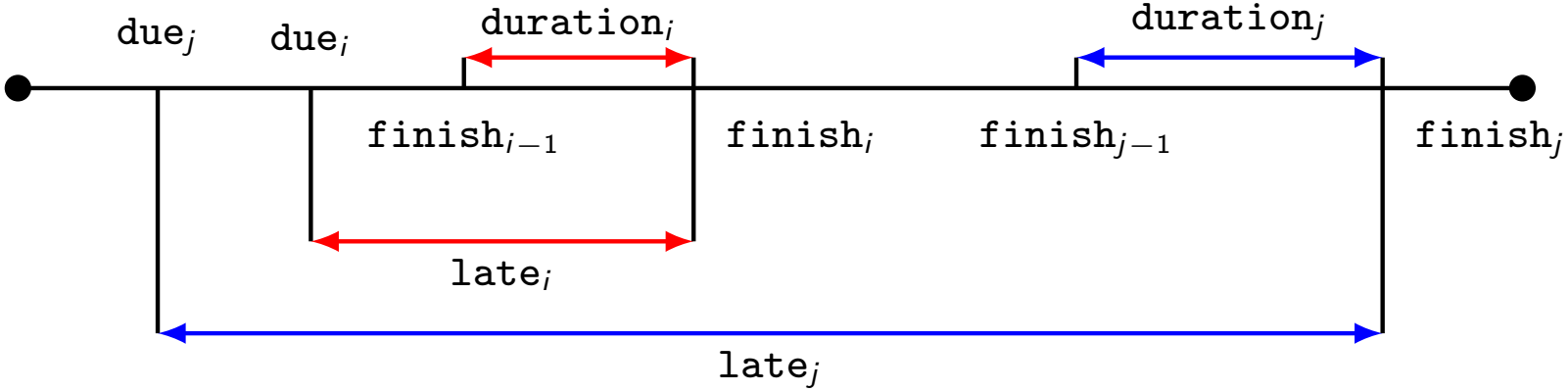
Actually, the durations have no effect on the optimal solution.

Solution

Schedule jobs in ascending order of deadlines, $\texttt{due}_i$.

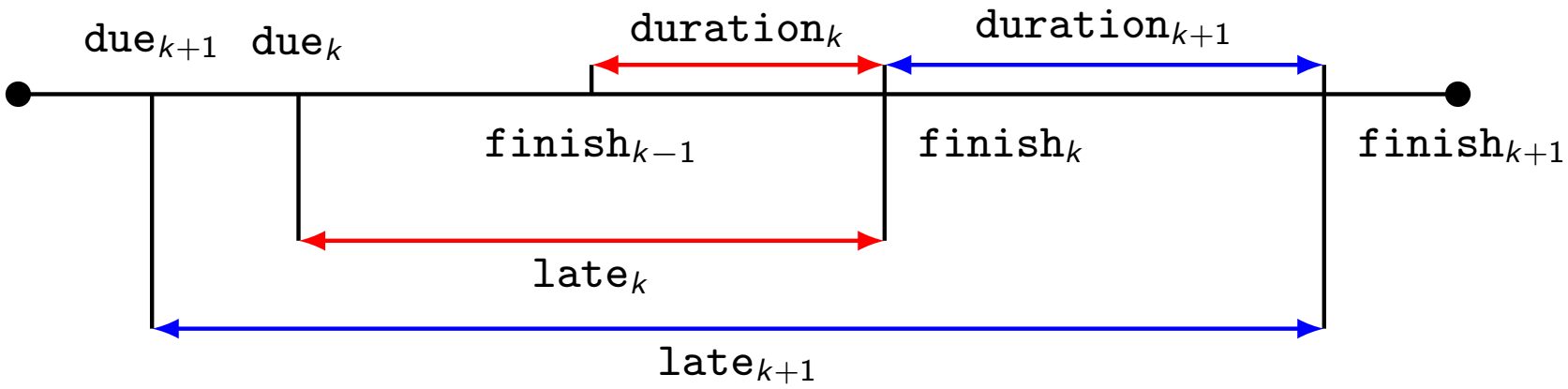
Proof of optimality.

- Assume you are given a solution ordering of jobs $A = [1, 2, 3, 4, ..., n]$.
- We say jobs i and j form an inversion if job i is scheduled before job j but is due after it (i.e. $\texttt{due}_j < \texttt{due}_i$).

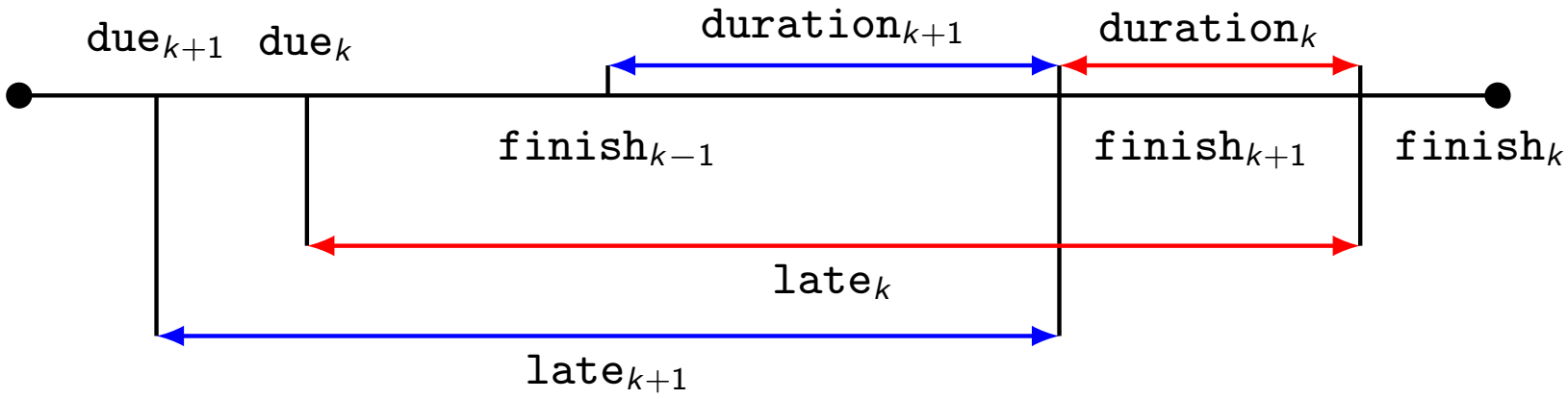


- Any sequence A that isn't sorted has an adjacent inversion.

- So there are two consecutive jobs $k, k+1$ in ordering A so that k is scheduled immediately before $k+1$, but $\texttt{due}_{k+1} < \texttt{due}_k$.



- What happens to the latenesses of k and $k+1$ when we swap their order?



- Are the latenesses of any other jobs changed by this swap? No

- Observe that swapping adjacent inverted jobs k and $k+1$ reduces the maximum lateness of the pair.
- The largest lateness overall has not increased as a result of this swap.
- Note: It would have been much harder to directly analyse <u>non-adjacent</u> inversions. Swapping non-adjacent jobs would also affect the finishing times and hence latenesses of every job in between them.

Table of Contents

1. Motivation
2. Optimal Selection
2.1 Activity selection
2.2 Cell tower placement
3. Detour: graph classes
4. Optimal Ordering
4.1 Tape storage
4.2 Minimising job lateness
5. Optimal Merging
5.1 The Huffman Code
6. Applications to graphs
6.1 Directed graph structure
6.2 Single source shortest paths
7. Matroids
7.1 Minimum spanning trees
8. Puzzle

Array Merging

- Assume you are given n sorted arrays of different sizes.
- You are allowed to merge any two arrays into a single new sorted array and proceed in this manner until only one array is left.

Exercise

Design an algorithm which achieves this task and moves array elements as few times as possible.

Give an informal justification why your algorithm is optimal.

This problem is somewhat related to the next problem, which is arguably among the most important applications of the greedy method!

Problem: Unambiguous code

You are given a set of symbols, for example the English alphabet plus some punctuation marks and a blank space (to be used between words), giving the 31 symbols:

abcdefghijklmnopqrstuvwxyz ,.?!

You want to encode these symbols using binary strings, so that sequences of such symbols can be decoded in an unambiguous way.

- "Unambiguous" means there is only one way to decode the binary string.

  original message $\rightarrow$ binary code $\rightarrow$ decoded message

- How long do these binary strings need to be?
- We have 31 symbols and $2^5 = 32 > 31$, so 5 bits per symbol?

A possible solution

Assign a different 5 bit sequence to each symbol in

abcdefghijklmnopqrstuvwxyz ,.?!

- This works! Every symbol can be encoded, and any message we encode can be decoded unambiguously.
- To decode you partition the bitstream into groups of 5 bits and use a lookup table.
- But this code is inefficient. Common symbols and rare symbols are all given binary encodings of the same length.
- It would be more efficient to encode <u>common</u> symbols with <u>short</u> strings.
- Is there a way to construct an efficient and unambiguous code?

Example

- Let's look at a smaller alphabet with just 6 symbols:

$$acekts$$

- Since $2^3 = 8 > 6$, we could assign each symbol a 3-bit binary string.
- But the letters e and t are very common in English, so let's give them shorter strings:

$$e = 00, t = 11$$

- And all the other letters get strings of length 3:

$$a = 010, c = 101, k = 111, s = 100$$

- Is this binary code unambiguous? No. Let's see why

Example - variable width encoding

Code: $a = 010, c = 101, e = 00, k = 111, s = 100, t = 11$
Original message: *cake*

$$\text{original message} \to \text{binary code} \to \text{decoded message}$$

Encoding step: $cake \to (101)ake \to (101)(010)ke \to (101)(010)(111)e \to (101)(010)(111)(00)$
Binary code. 10101011100
Decoding step. **ca111**00
Two possible decodings:

- 101 010 11 100 $\to$ *cats*
- 101 010 111 00 $\to$ *cake*

Our code was ambiguous because the encoding of $t = 11$ is a <u>prefix</u> of $k = 111$.

- A code in which no coded symbol is a prefix of another is called a **prefix code**.
- Prefix codes are unambiguous.

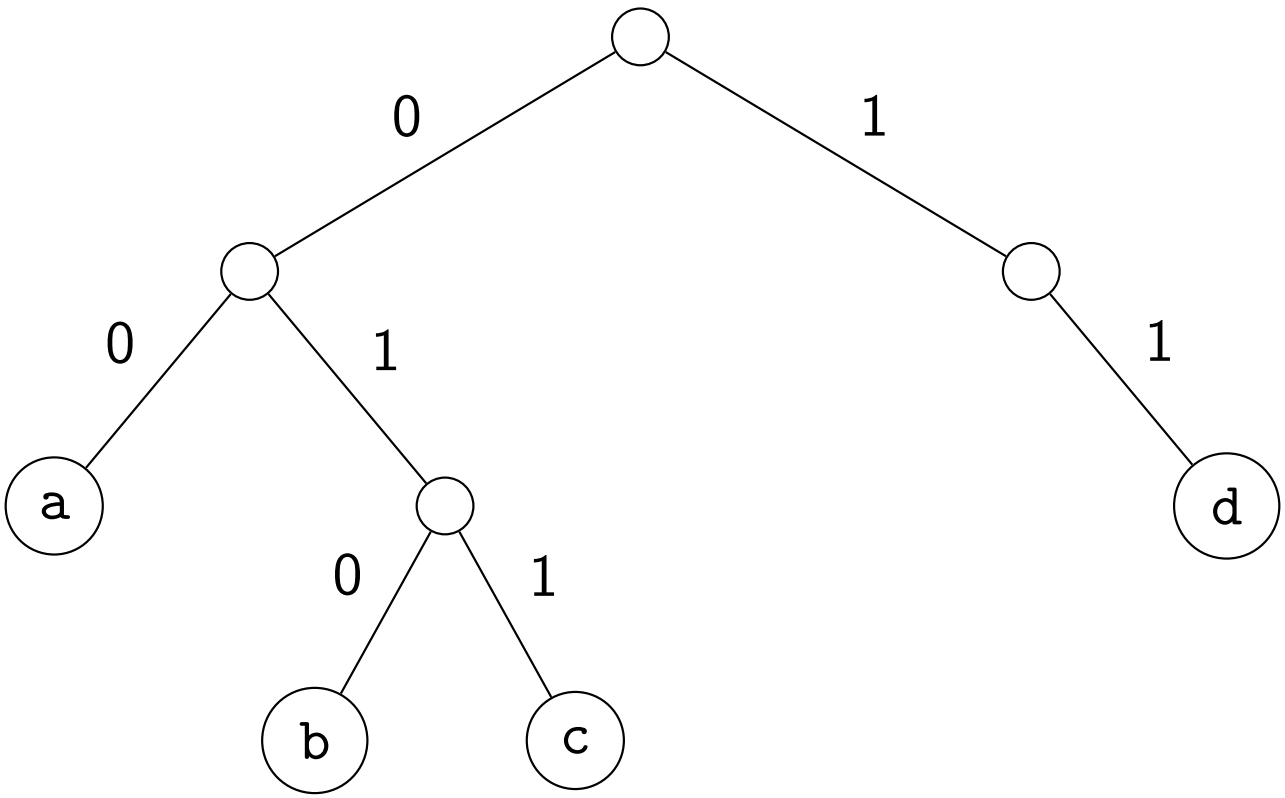
Problem: efficient, unambiguous code

You are given a set of symbols, for example the English alphabet plus punctuation marks and a blank space (to be used between words), giving the 31 symbols:

abcdefghijklmnopqrstuvwxyz ,.?!

Each symbol has a frequency (probability of occurring). You want to encode these symbols using binary strings, so that sequences of such symbols can be decoded in an unambiguous way. And so that the expected length of the encoded text is minimised.

- Any encoding into binary can be visualised using a binary tree, where left branches represent ‘0’ and right branches ‘1’.
- In a prefix code, no ancestor of a codeword is itself a codeword, so all codewords must be located at leaves.



Optimal solution

Huffman code!

- Huffman's method uses a greedy algorithm to construct an optimal prefix code.
- This makes sure that the average number of bits in an encoding of an "average" text is as small as possible.
- We'll next go through an example of how to apply Huffman's code. For the details of the algorithm, and the proof of optimality, see the textbook.
- MATH3411 Information, Codes and Ciphers covers this and much more!

Table of Contents

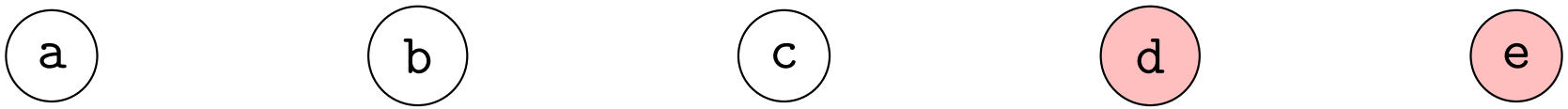
1. Motivation
2. Optimal Selection
2.1 Activity selection
2.2 Cell tower placement
3. Detour: graph classes
4. Optimal Ordering
4.1 Tape storage
4.2 Minimising job lateness
5. Optimal Merging
5.1 The Huffman Code
6. Applications to graphs
6.1 Directed graph structure
6.2 Single source shortest paths
7. Matroids
7.1 Minimum spanning trees
8. Puzzle

Main idea.

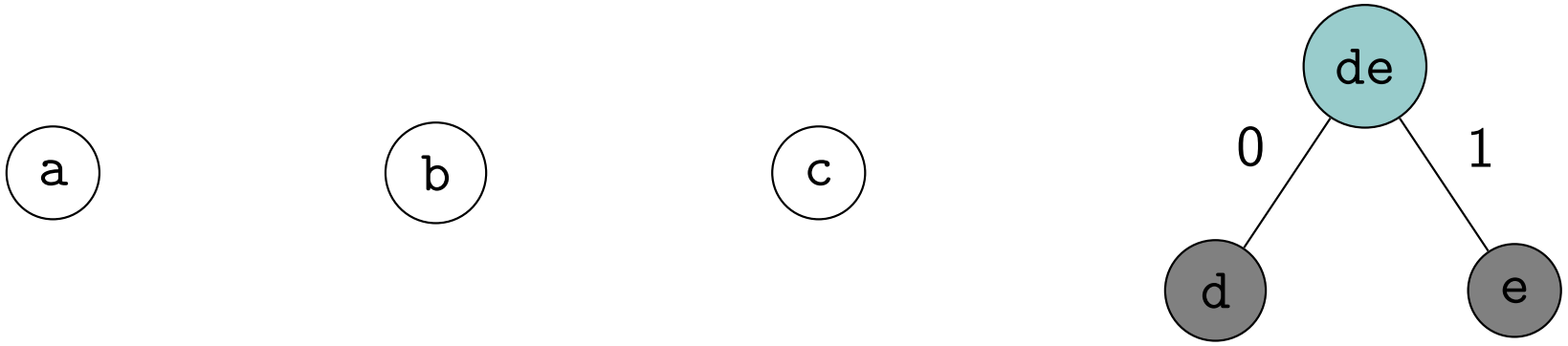
- Make a priority queue of symbols, each weighted by its frequency.
- Repeatedly combine the two lowest frequency symbols.
  - Remove the two symbols of lowest frequency, say $\mathtt{x}$ and $\mathtt{y}$ with frequencies $f_{\mathtt{x}}$ and $f_{\mathtt{y}}$.
  - Make a new symbol $\mathtt{xy}$ with frequency $f_{\mathtt{x}} + f_{\mathtt{y}}$, and insert it to the priority queue.
- As we do this, we maintain a forest, initially consisting of n isolated vertices.
  - When symbols $\mathtt{x}$ and $\mathtt{y}$ are combined, we add a new vertex representing $\mathtt{xy}$, whose left and right children are $\mathtt{x}$ and $\mathtt{y}$.
  - This records that the codeword for $\mathtt{x}$ is to be found by appending a '0' to the codeword for $\mathtt{xy}$, and likewise for $\mathtt{y}$ by appending a '1'.

The Huffman Code: Example

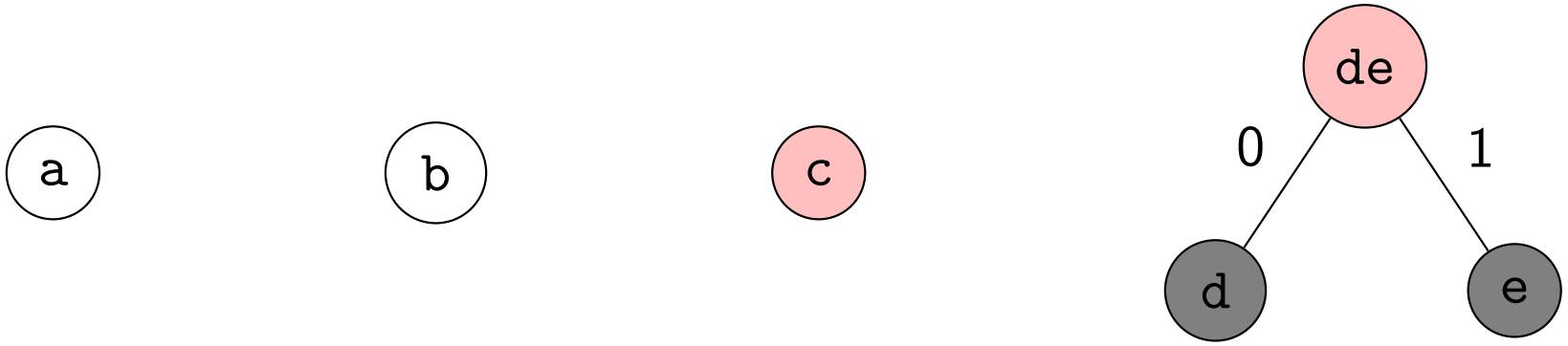
symbol	a	b	c	d	e
frequency	0.32	0.25	0.20	0.18	0.05



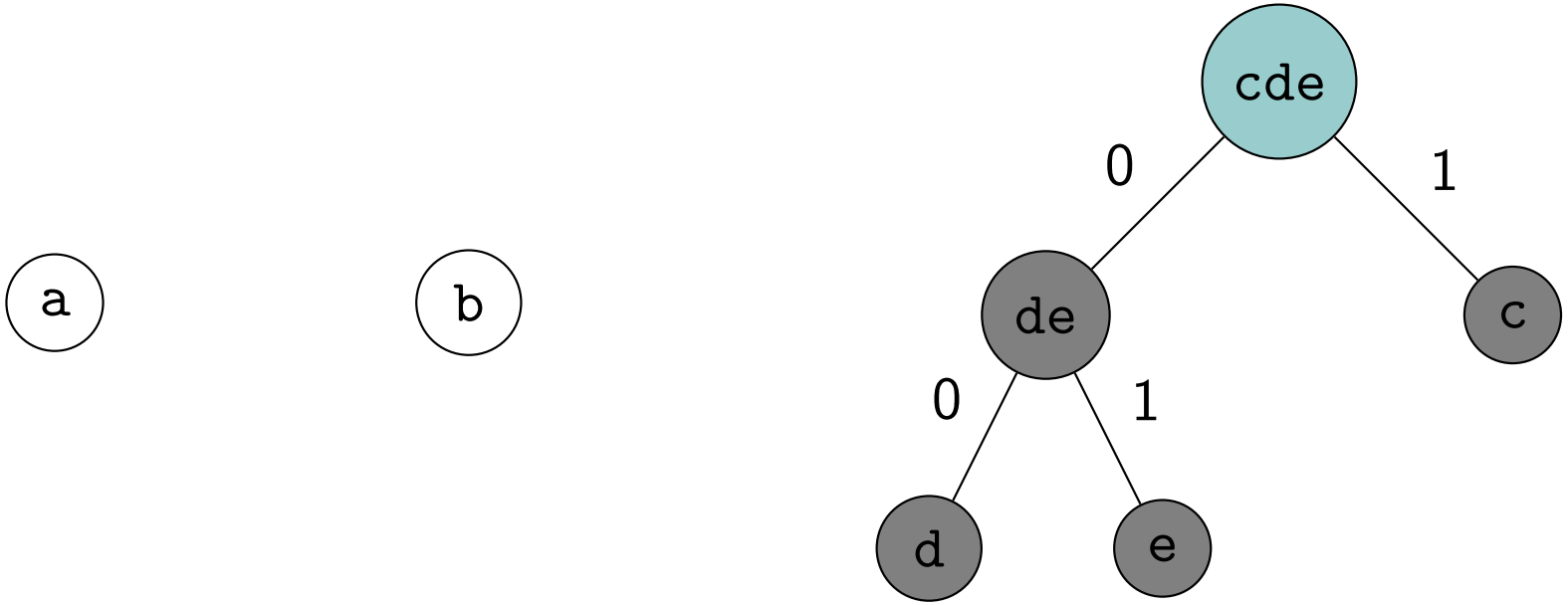
symbol	a	b	c	d	e	de
frequency	0.32	0.25	0.20	0.18	0.05	0.23



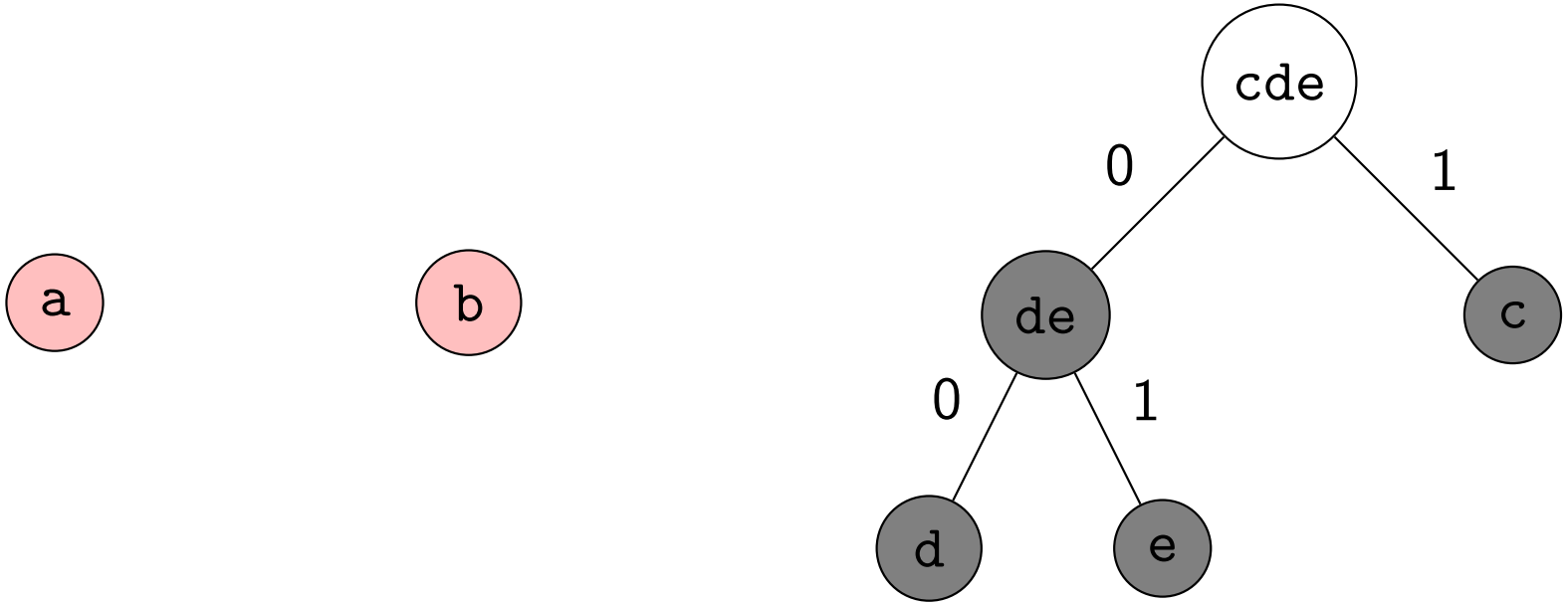
symbol	a	b	c	de
frequency	0.32	0.25	0.20	0.23



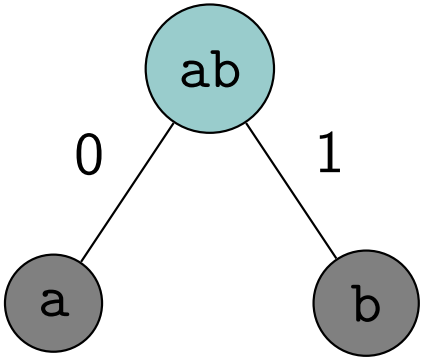
symbol	a	b	c	de	cde
frequency	0.32	0.25	0.20	0.23	0.43

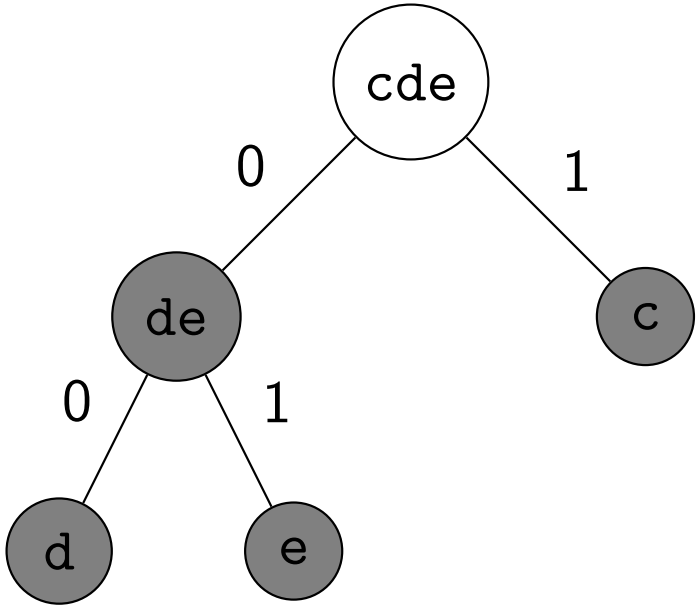


symbol	a	b	cde
frequency	0.32	0.25	0.43

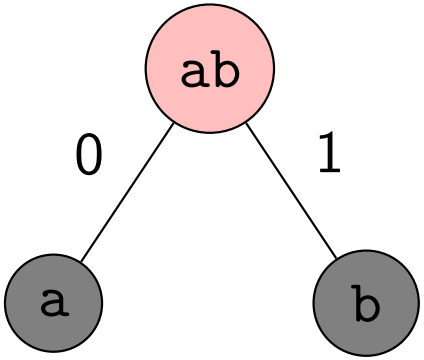


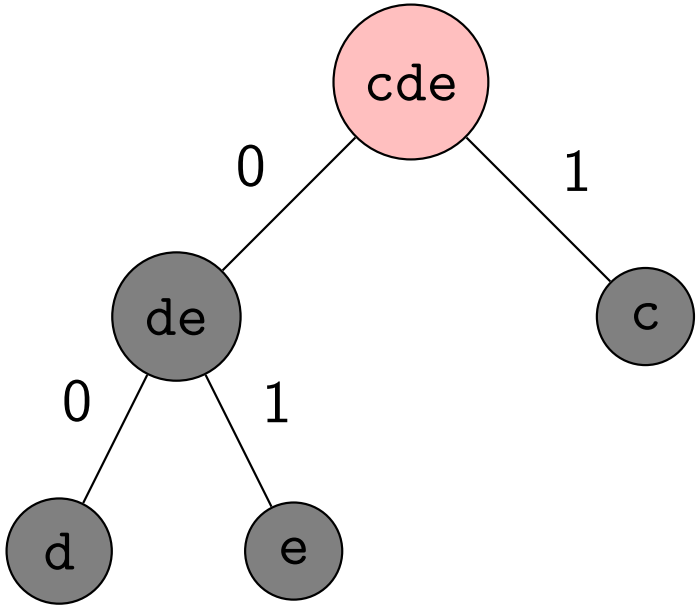
symbol	a	b	ab	cde
frequency	0.32	0.25	0.57	0.43



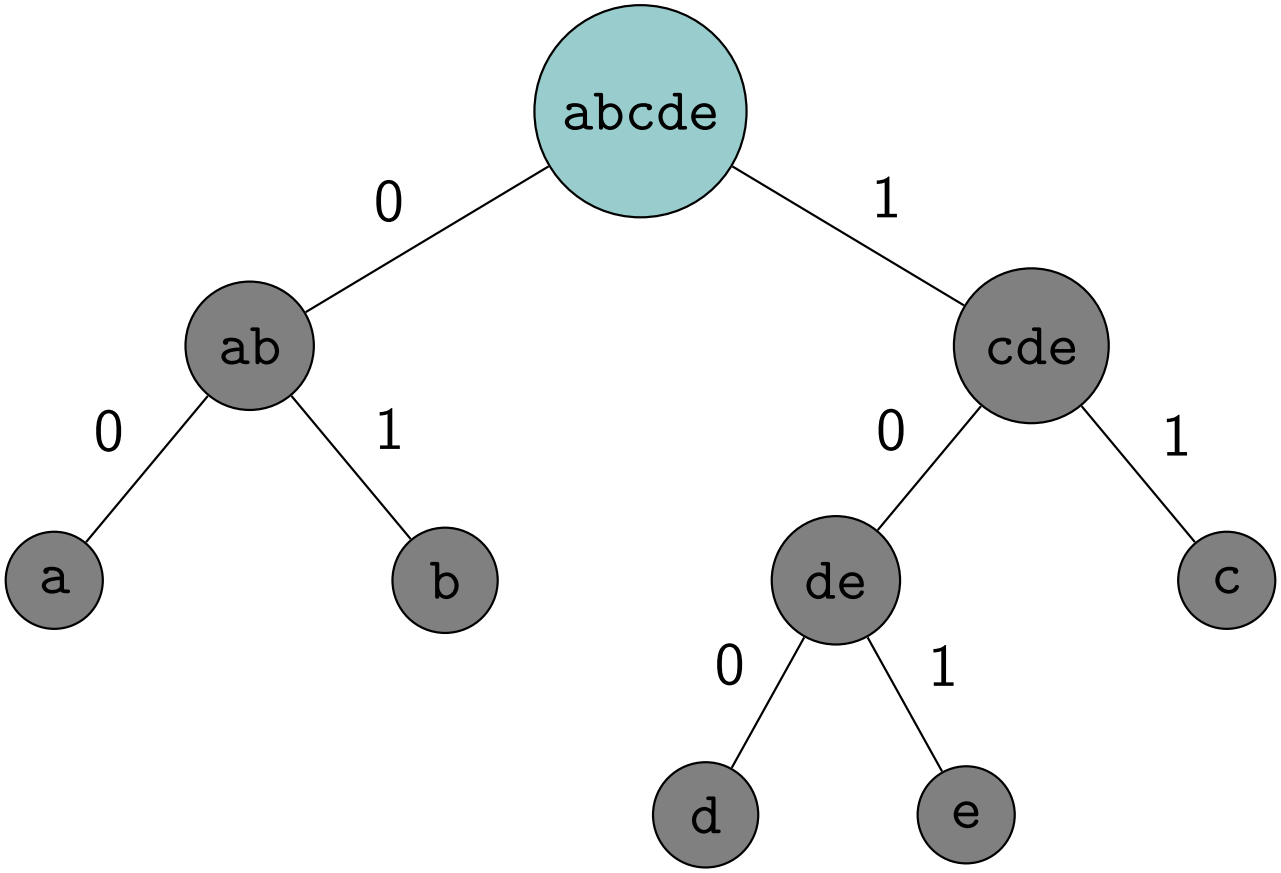


symbol	ab	cde
frequency	0.57	0.43



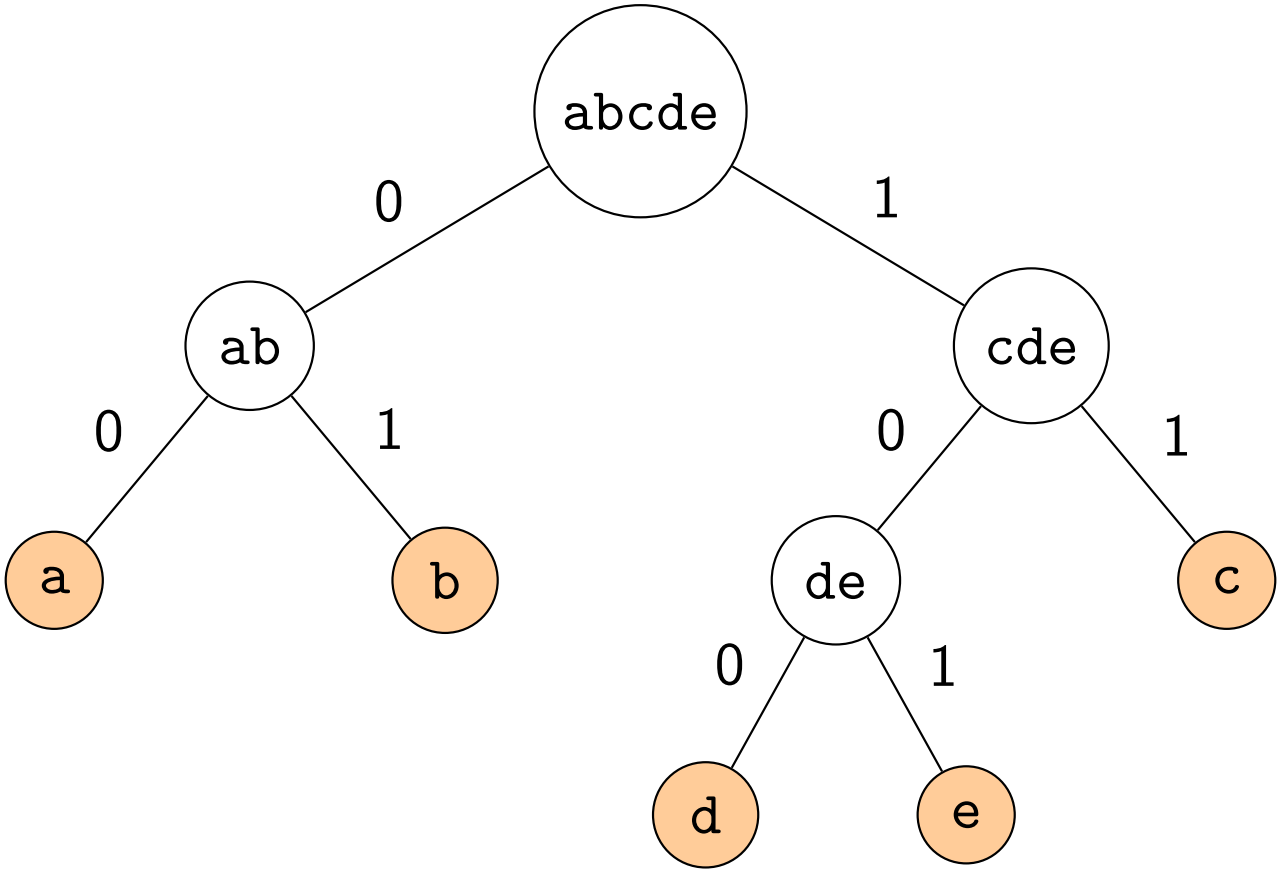


symbol	cde	ab	abcde
frequency	0.43	0.57	1.00



The Huffman Code: Example

symbol	a	b	c	d	e
codeword	00	01	11	100	101



symbol i	a	b	c	d	e
codeword C_i	00	01	11	100	101
frequency f_i	0.32	0.25	0.20	0.18	0.05

Average codeword length is

$$
\begin{aligned}
L &= \sum_{i \in \{\mathtt{a},\mathtt{b},\mathtt{c},\mathtt{d},\mathtt{e}\}} \mathrm{len}(C_i) \times f_i \\
&= [\mathrm{len}(C_{\mathtt{a}}) \times f_{\mathtt{a}}] + [\mathrm{len}(C_{\mathtt{b}}) \times f_{\mathtt{b}}] + [\mathrm{len}(C_{\mathtt{c}}) \times f_{\mathtt{c}}] \\
&\quad + [\mathrm{len}(C_{\mathtt{d}}) \times f_{\mathtt{d}}] + [\mathrm{len}(C_{\mathtt{e}}) \times f_{\mathtt{e}}] \\
&= 2 \times 0.32 + 2 \times 0.25 + 2 \times 0.20 + 3 \times 0.18 + 3 \times 0.05 \\
&= 2.23
\end{aligned}
$$

Lemma

Given the frequency f_i of each symbol i, the Huffman code produces an optimal prefix code.

1. First, we prove that an optimal prefix code must arise from a full binary tree; one in which every node has exactly zero or two children.

   - Hopefully obvious; a node with only one child could be replaced by that child.

2. Then, we prove that in some optimal tree, the two symbols of lowest frequency are siblings at the greatest depth.
   - If these two symbols aren't siblings at the greatest depth, then swap them with whatever symbols are.
     - Either nothing changes depth, or
     - less frequent symbols increase in depth and more frequent symbols decrease in depth by the same amount, so it is a net improvement.

3. Finally, we prove (by induction on the number of symbols) that the Huffman code is optimal.

   - Perform the combine operation on some optimal tree and the Huffman tree to get trees on $n - 1$ vertices, and use the induction hypothesis.

Table of Contents

1. Motivation
2. Optimal Selection
2.1 Activity selection
2.2 Cell tower placement
3. Detour: graph classes
4. Optimal Ordering
4.1 Tape storage
4.2 Minimising job lateness
5. Optimal Merging
5.1 The Huffman Code
6. Applications to graphs
6.1 Directed graph structure
6.2 Single source shortest paths
7. Matroids
7.1 Minimum spanning trees
8. Puzzle

Table of Contents

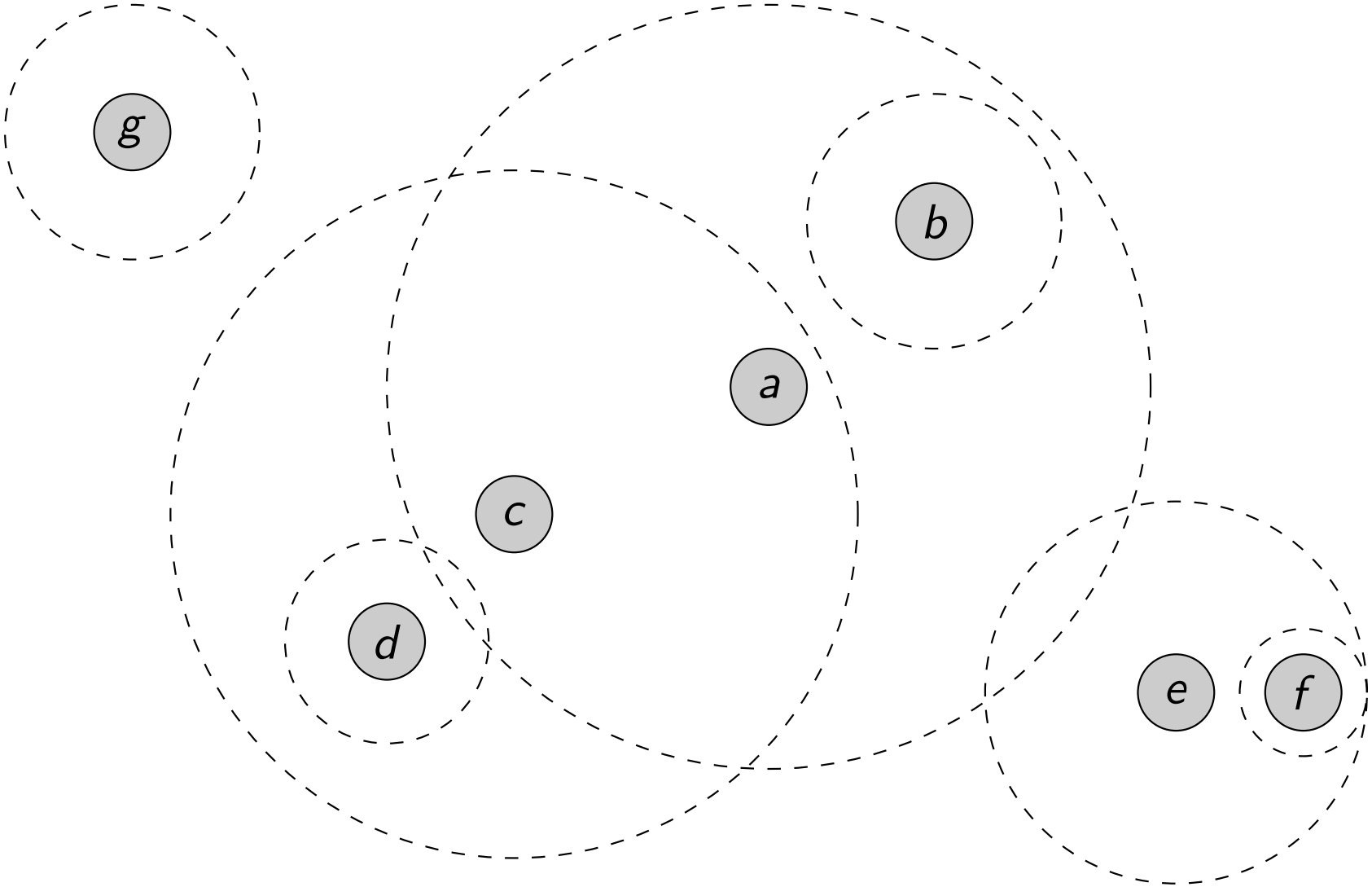
1. Motivation
2. Optimal Selection
2.1 Activity selection
2.2 Cell tower placement
3. Detour: graph classes
4. Optimal Ordering
4.1 Tape storage
4.2 Minimising job lateness
5. Optimal Merging
5.1 The Huffman Code
6. Applications to graphs
6.1 Directed graph structure
6.2 Single source shortest paths
7. Matroids
7.1 Minimum spanning trees
8. Puzzle

Problem

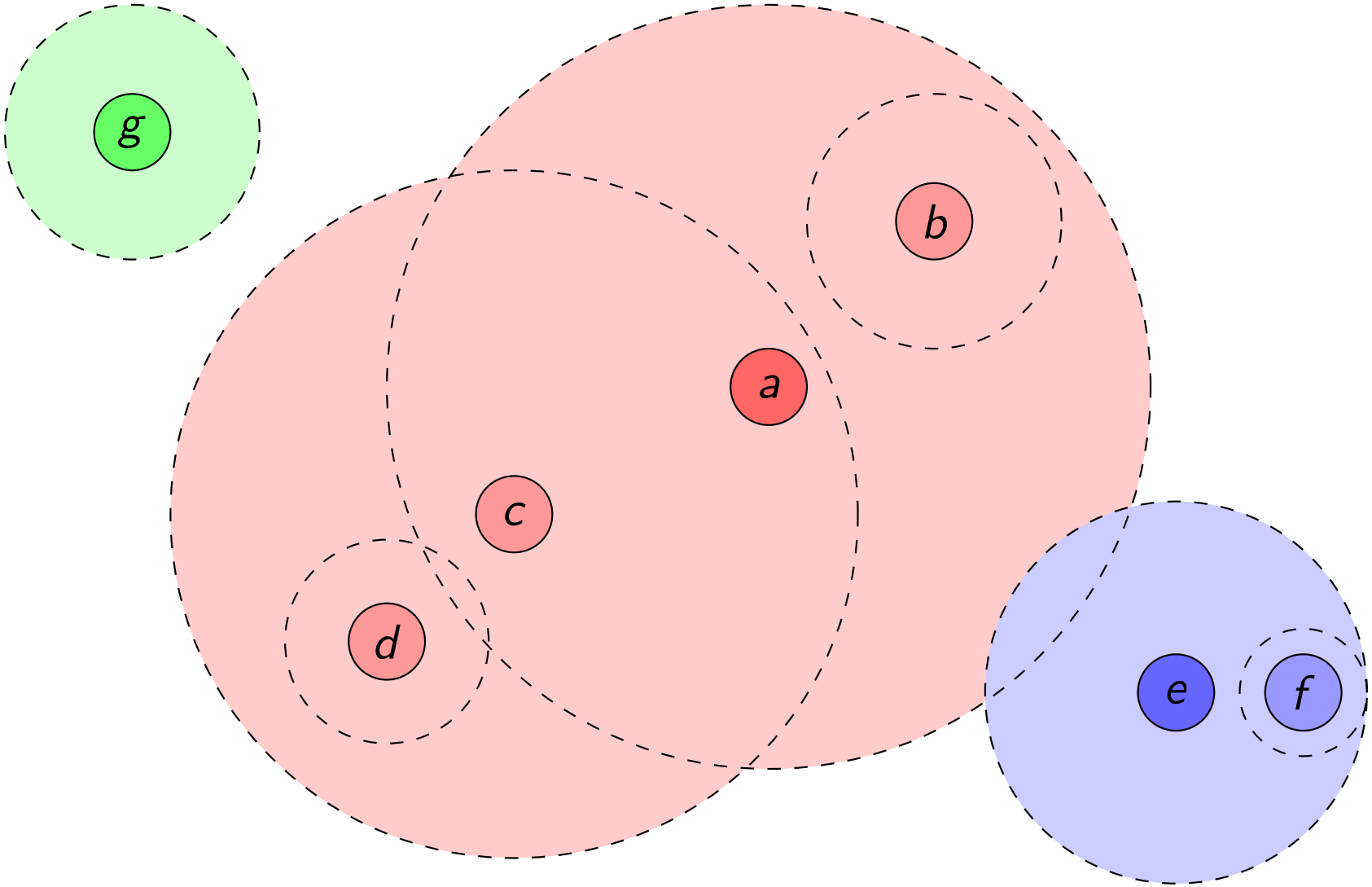
Instance: There are n radio towers for broadcasting tsunami warnings. You are given the (x_i, y_i) coordinates of each tower i and its radius r_i of range. When a tower is activated, all towers within the range of the tower will also activate, and those can cause other towers to activate and so on.

You need to equip <u>some</u> of these towers with seismic sensors. When these sensors are activated, they activate the tsunami warning at their tower. These activated towers then activate all towers in their range, and so on.

Task: Design an algorithm which finds the fewest number of towers you must equip with seismic sensors so that all towers become activated.


g
b
a
c
d
e
f

Tsunami Warning



How to construct a greedy algorithm for placing sensors?

What inputs were we given?

- Locations (x_i, y_i)
- radii r_i.

What did we learn from the example?

- Towers with very small radius could not activate others. So big radius is helpful.
- Towers can only directly activate towers whose locations are inside their radius. So maybe location is also important?
- A tower i can indirectly activate another tower j if there is a sequence of activations from i to j. Is there a way to identify when one tower can indirectly activate another?

Attempt 1 (only consider radius)

Find the unactivated tower with the largest radius (breaking ties arbitrarily), and place a sensor at this tower. Find and remove all towers activated as a result. Repeat.

Attempt 2 (only consider direct activations)

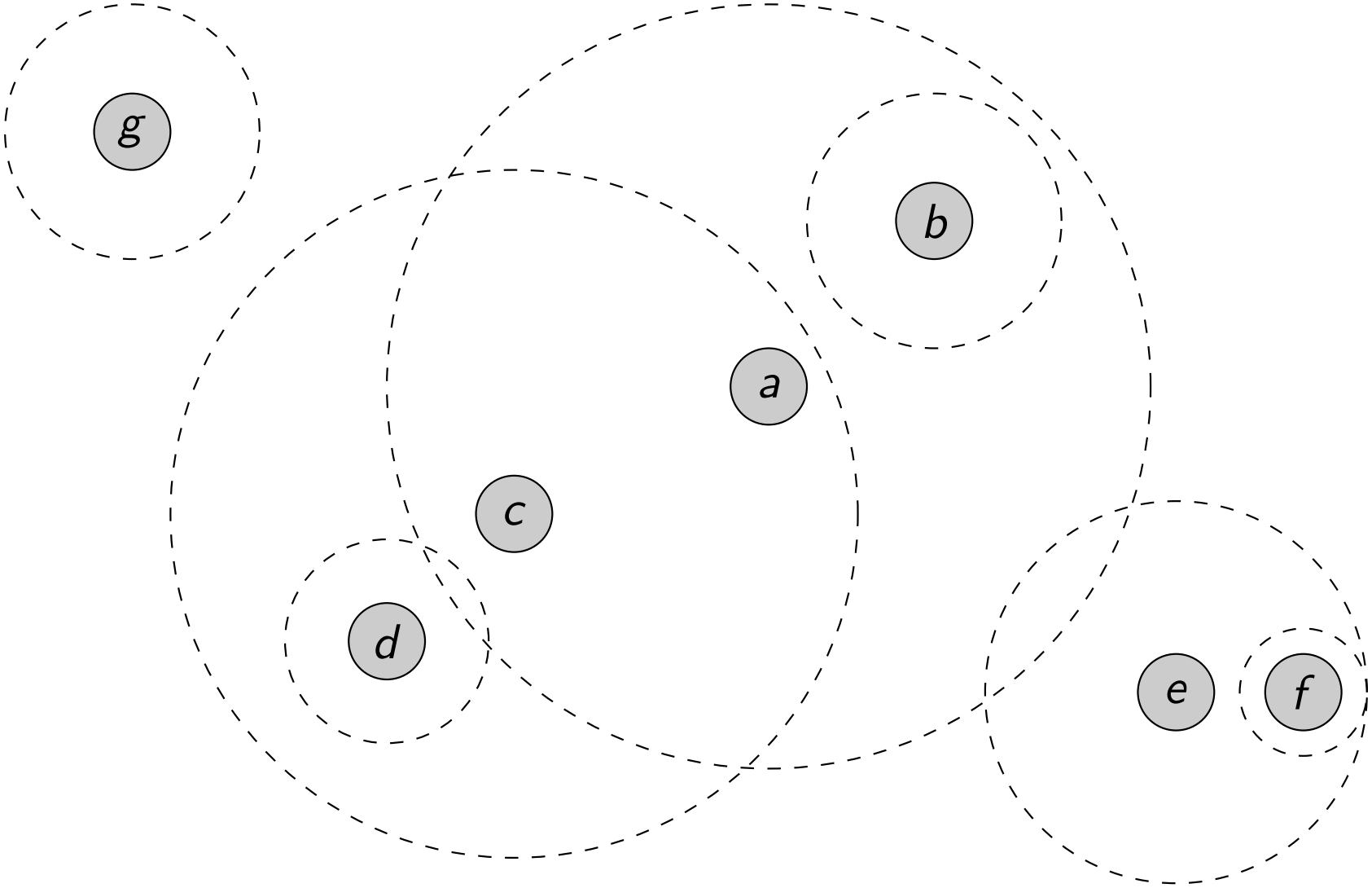
Find the unactivated tower with the largest number of towers within its range (breaking ties arbitrarily), and place a sensor at this tower. Find and remove all towers activated as a result. Repeat.

Exercise

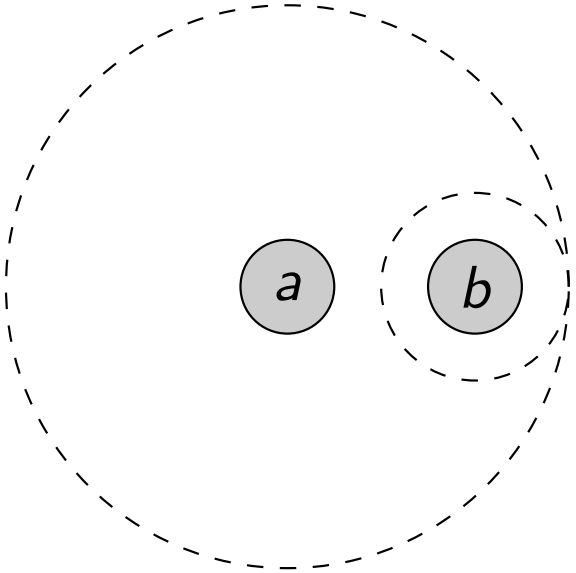
Give examples which show that neither of these algorithms solve the problem correctly.

Our greedy algorithm also needs to consider indirect activations. How?

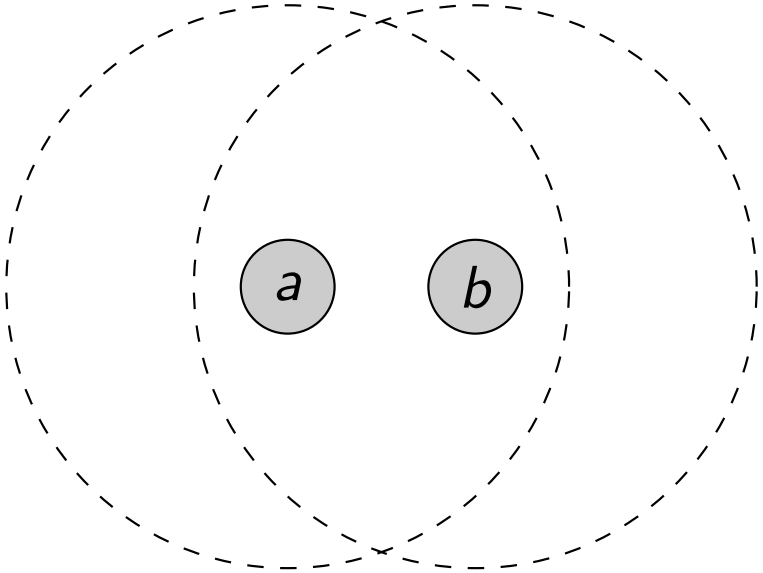
Maybe we can use a graph?



It is useful to consider the towers as vertices of a directed graph, where an edge from tower a to tower b indicates that the activation of a directly causes the activation of b, that is, b is within the radius of a.



$a \to b$



$a \leftrightarrow b$

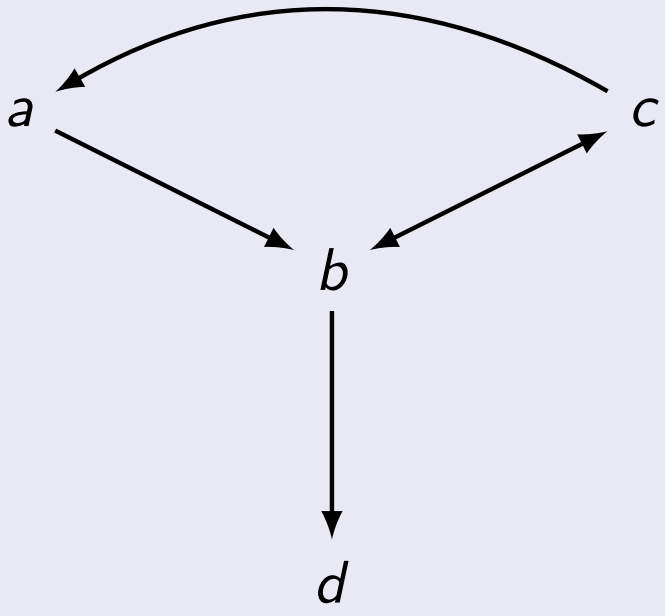
Observation

Suppose that activating tower a causes tower b to also be activated, and vice versa. Then we never want to place sensors at both towers; indeed, placing a sensor at a <u>is equivalent to</u> placing a sensor at b.

How can we can extend this notion to a larger number of towers?

- Cycles also have this property.
- Can we do better?

Example



All four towers can be activated by placing just one sensor at a or b or c.

Observation

Let S be a subset of the towers such that that activating <u>any</u> tower in S causes the activation of <u>all</u> towers in S.

We never want to place more than one sensor in S, and if we place one, then it doesn't matter where we put it.

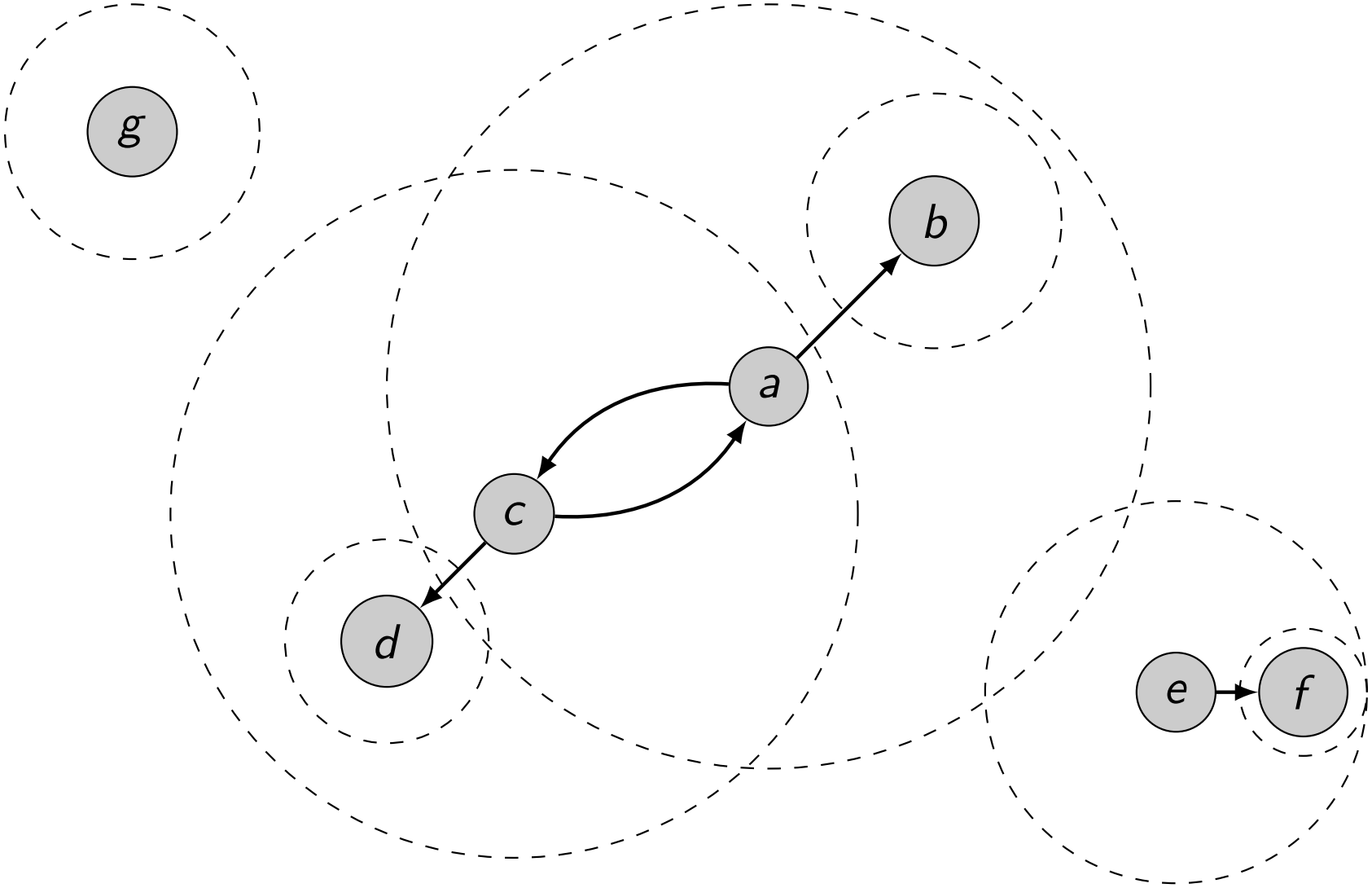
In this way, we can treat all of S as a unit; a "super-tower".

Definition

Given a directed graph $G = (V, E)$ and a vertex v, the strongly connected component of G containing v consists of all vertices $u \in V$ such that there is a directed path in G from v to u and a directed path from u to v. We will denote it by C_v.

In the terms of our problem, strongly connected components are maximal super-towers.

What are the strongly-connected components below?



How do we find the strongly connected component $C_v \subseteq V$ containing v?

Claim

u is in C_v if and only if u is reachable from v and v is reachable from u.

We can split this into two subproblems:

- Find all the vertices that can be reached from v.
- Find all the vertices which can reach v.

Then a vertex u is in C_v if and only if it appears in both of these sets.

To find all the vertices that can be reached from v:

- Do a BFS or DFS of G to find the set R_v of vertices of the directed BFS/DFS tree rooted at v.

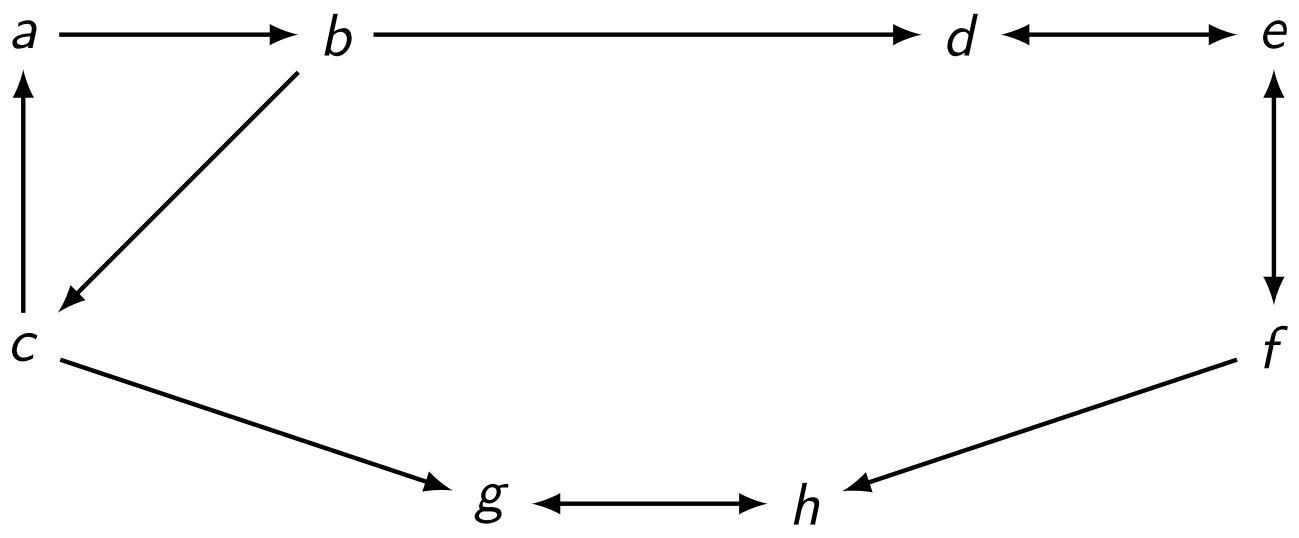
Finding the vertices that can reach v is harder:

- Construct another graph $G_{rev} = (V, E_{rev})$ consisting of the same set of vertices V but with the set of edges E_{rev} obtained by reversing the direction of all edges E of G.
- Find the set R'_v of vertices of the directed BFS/DFS tree of G_{rev} rooted at v.

Claim

u is in C_v if and only if u is reachable from v and v is reachable from u.
In other words, u is reachable from v in both G and G_{rev}.
Equivalently, u is in both R_v and R'_v.

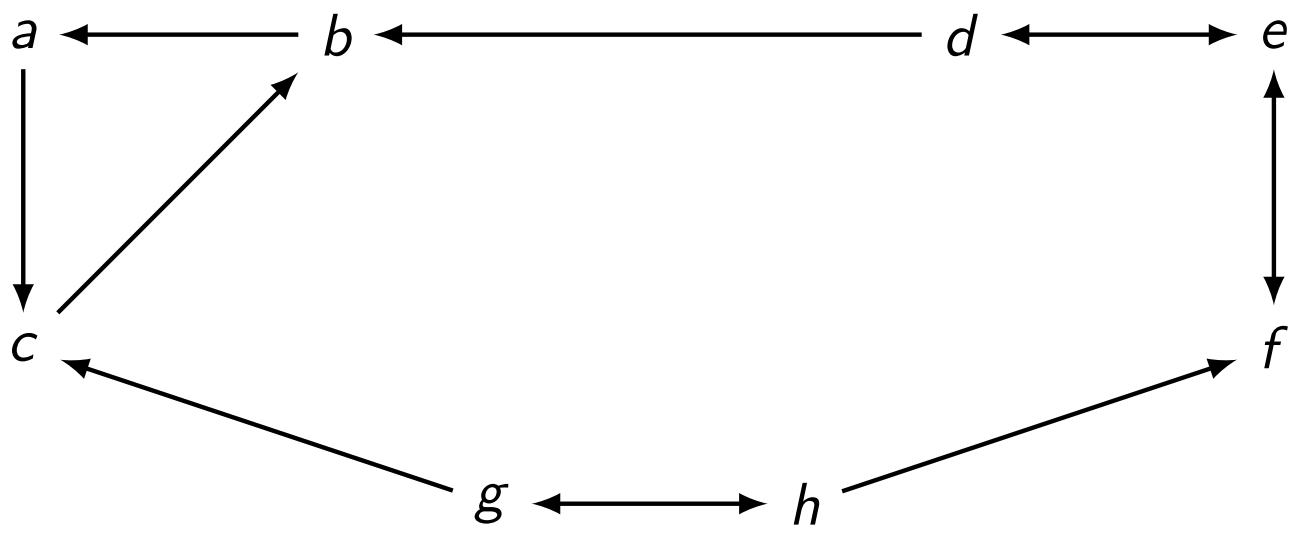
Suppose the original graph $G = (V, E)$ is



Then the set of vertices reachable from e is

$$R_e = \{d, e, f, g, h\}.$$

The reverse graph $G_{rev} = (V, E_{rev})$ is



Then the set of vertices reachable from e is

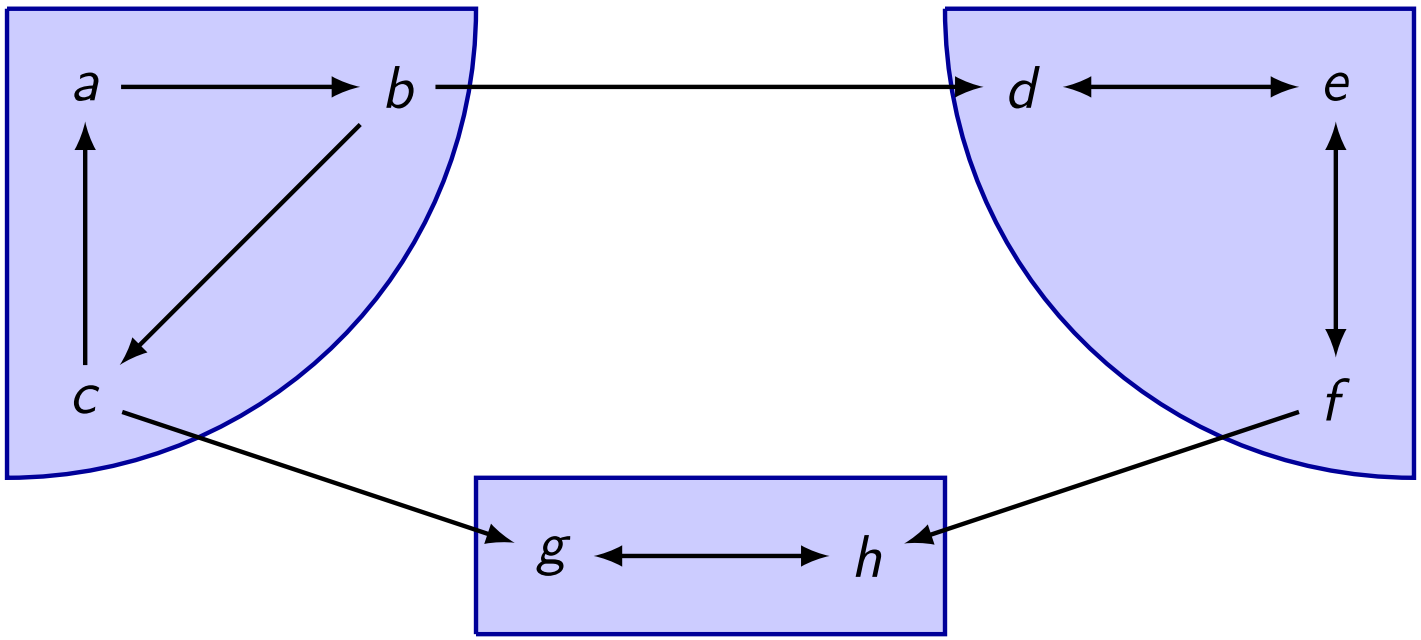
$$R'_e = \{a, b, c, d, e, f\}.$$

- Combining
$$R_e = \{d, e, f, g, h\}$$
with
$$R'_e = \{a, b, c, d, e, f\},$$
we have
$$C_e = R_e \cap R'_e = \{d, e, f\}.$$
- Note that C_d and C_f are also the same set, namely $\{d, e, f\}$.
- Similarly, we find that $C_a = \{a, b, c\}$ and $C_g = \{g, h\}$.

To recap:

- Use BFS to find the set $R_v \subseteq V$ of all vertices in V which are reachable in G from v.
- Similarly find the set $R'_v \subseteq V$ of all vertices which are reachable in G_{rev} from v.
- The strongly connected component of G containing v is given by $C_v = R_v \cap R'_v$.

Therefore the decomposition of G into strongly connected components is



Time complexity.

- Finding all strongly connected components in this way could require $O(|V|)$ traversals of the graph.
- Each of these traversals is a BFS, requiring $O(|V| + |E|)$ time.
- Therefore the total time complexity is $O(|V|(|V| + |E|))$.
- Can we do better?
- Could one DFS suffice?

Tarjan's Algorithm

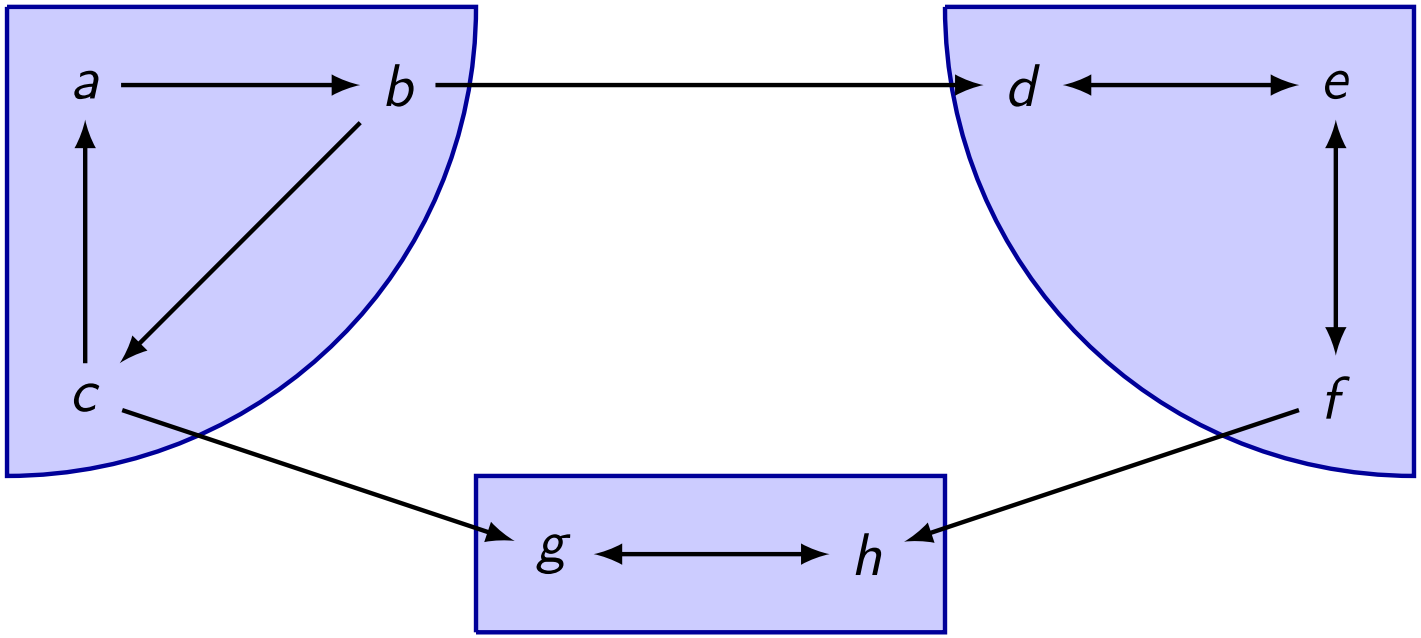
- Do a regular DFS on the graph, but with an additional stack.
- The DFS will assign an ID from 0 to $n-1$ to each vertex, in the order they are encountered. It makes sure to visit each vertex, starting again at an unvisited vertex as long as there are unvisited vertices in the graph.
- Store a value LOW for each vertex, which is initially its ID and will be the smallest ID of a vertex that can be reached by following the edges of the DFS subtree and a single back-edge.
- When a new vertex is visited, push it onto the stack, mark it as "in-stack", and unmark it as such when it is popped. Set its ID and LOW values.

- When we encounter a back-edge (u, v) to a vertex v that is "in-stack", we know that each vertex after v on the stack
  - can be reached from v (following the edges of the DFS), and
  - can reach v (using the edge we just encountered),

  so they all belong to the same strongly connected component as v. Vertex u updates its LOW value to the ID of v.
- The DFS continues as normal, without following back-edges.
- On backtracking from a child to a parent in the DFS tree, the LOW value of the parent becomes the min of the LOW values of the child and the parent.
- After having visited all neighbours of a vertex v, if v's LOW value equals its ID, then pop all vertices from the stack up to and including this vertex; these form one strongly connected component.
- The time complexity is $O(|V| + |E|)$.

- Kosaraju's algorithm (presented in CLRS §22.5) is an alternative.
  - It performs a DFS in G where it records the finishing times of each vertex once it is fully processed,
  - and a DFS in G_{rev} considering nodes in decreasing order of their finish times in the first DFS.
- It also runs in <u>linear time</u>, i.e., $O(|V| + |E|)$.
- A linear time algorithm is asymptotically "no slower than" reading the graph, so we can run these algorithms "for free", i.e., without worsening the time complexity of our solution to a problem.

But what should we do next?



- We've found a decomposition into strongly connected components.
- How do we use this to choose where to place the sensors?

- It should be clear that distinct strongly connected components are disjoint sets, so the strongly connected components form a partition of V.
- Let C_G be the set of all strongly connected components of a graph G.

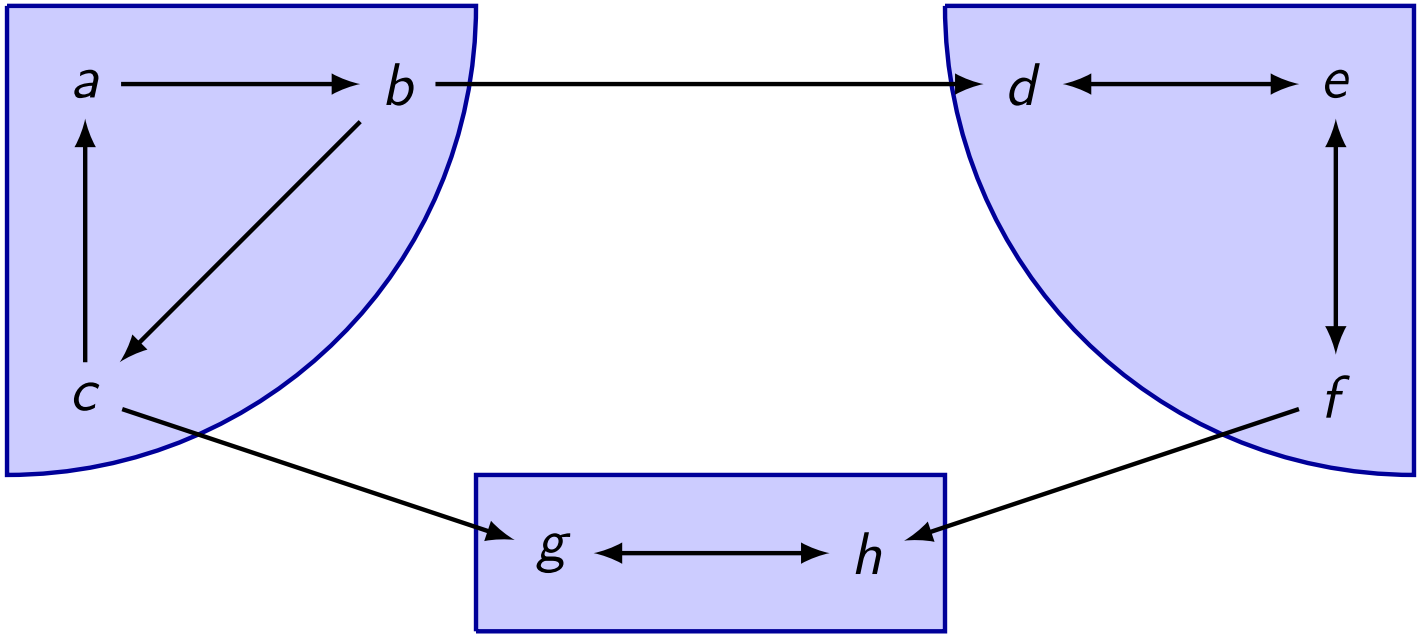
Definition

Define the <u>condensation graph</u> $\Sigma_G = (C_G, E^*)$, where

$$E^* = \{(C_u, C_v) : (u, v) \in E,\ C_u \neq C_v\}.$$

The vertices of Σ_G are the strongly connected components of G, and the edges of Σ_G correspond to those edges of G that are not within a strongly connected component, with duplicates ignored.

Recall our earlier example



The condensation graph is simply

$$\begin{tikzcd}
\{a, b, c\} \arrow[rr] \arrow[dr] & & \{d, e, f\} \arrow[dl] \\
& \{g, h\} &
\end{tikzcd}$$

If this were our tsunami problem,

- Where would we place sensors on the condensation graph?
- Why?

Let's recap:

- We begin our solution to the tsunami warning problem by finding the condensation graph.
- Now we have the set of super-towers (strongly connected components), and we know for each super-tower (SCC) which others it can activate.
- Our task is to decide which super-towers need a sensor installed in order to activate all the super-towers.
- We need to know one more property about the condensation graph.

Claim

The condensation graph Σ_G is a directed acyclic graph.

Proof Outline

Suppose there is a cycle in Σ_G. Then the vertices on this cycle are not <u>maximal</u> strongly connected sets, as they can be merged into an even larger strongly connected set.

Solution

Place a sensor in each super-tower without incoming edges in the condensation graph.

Proof

These super-towers cannot be activated by another super-tower, so they each require a sensor. This shows that there is no solution using fewer sensors.

Proof (continued)

We still have to prove that this solution activates all super-towers.

Consider a super-tower with one or more incoming edges. Follow any of these edges backwards, and continue backtracking in this way.

Since the condensation graph is acyclic, this path must end at some super-tower without incoming edges. The sensor placed here will then activate all super-towers along our path.

Therefore, all super-towers are activated as required.

So the steps we used to solve the tsunami problem were:

Algorithm

- **Input:** towers i with locations (x_i, y_i) and radii r_i.
- Construct a directed graph G, where (u, v) is an edge if tower u can directly activate tower v.
- Construct the reversed graph G_{rev} of G.
- Use DFS or BFS on G and G_{rev} to find the strongly connected components of G.
- Form the condensation graph Σ_G, whose vertices are the strongly connected components of G.
- **Solution:** Place a sensor on every vertex of Σ_G that has no incoming edges.

Why is this a greedy algorithm? Here's one answer...

- We choose to place a sensor on a vertex v whose strongly-connected component C_v has the smallest number of incoming edges in Σ_v.

- It just so happens that this smallest number of incoming edges is always 0.

A more compelling answer is that we place a sensor on the first vertex of a topological sort of the condensation graph.

If a directed graph G has no cycles (e.g., when G is a condensation graph), then we can order the vertices of G by the following rule:

Definition

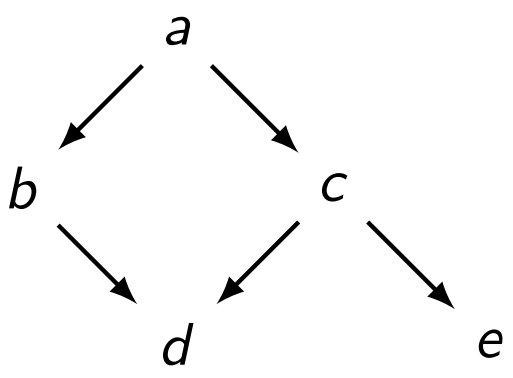
Let $G = (V, E)$ be a directed acyclic graph, and let $n = |V|$. A <u>topological sort</u> of G is a linear ordering (permutation) of its vertices $\sigma : V \to \{1, \ldots, n\}$ such that if there exists an edge $(v, w) \in E$ then v precedes w in the ordering, i.e., $\sigma(v) < \sigma(w)$.

Property

A directed acyclic graph permits a topological sort of its vertices.

Note that the topological sort is not necessarily unique, i.e., there may be more than one valid topological ordering of the vertices.

Example. Find a topological sort of the vertices in this graph:



There are many solutions, such as:

- *abcde*
- *acebd*

Why is *abdce* not a solution? This is because $\overrightarrow{cd}$ is an edge, but d appears before c in the list.

How do we topologically sort the vertices?

Algorithm

Maintain:

- a list L of vertices, initially empty,
- an array D consisting of the in-degrees of the vertices, and
- a set S of vertices with no incoming edges.

Algorithm (continued)

While set S is non-empty, select a vertex u of with $D[u] = 0$ in the set.

- Remove it from S and append it to L.
- Then, for every outgoing edge $e = (u, v)$ from this vertex, remove the edge from the graph, and decrement $D[v]$ accordingly.
  - If $D[v]$ is now zero, insert v into S.

If there are no edges remaining, then L is a topological ordering. Otherwise, the graph has a cycle.

Time complexity.

- This algorithm runs in $O(|V| + |E|)$, that is, linear time.
- Once again, we can run this algorithm "for free" as it is asymptotically no slower than reading the graph.
- In problems involving directed acyclic graphs, it is often useful to start with a topological sort and then think about the actual problem!
- A topological ordering is often a natural way to process the vertices. We'll see more of this in Dynamic Programming.

Table of Contents

1. Motivation
2. Optimal Selection
2.1 Activity selection
2.2 Cell tower placement
3. Detour: graph classes
4. Optimal Ordering
4.1 Tape storage
4.2 Minimising job lateness
5. Optimal Merging
5.1 The Huffman Code
6. Applications to graphs
6.1 Directed graph structure
6.2 Single source shortest paths
7. Matroids
7.1 Minimum spanning trees
8. Puzzle

Problem

Instance: a directed graph $G = (V, E)$ with non-negative edge-weights $w(e)$, and a designated source vertex $s \in V$.

We will assume that for every $v \in V$ there is a path from s to v.

Task: find the weight of the shortest path from s to v for every $v \in V$.

Example. Suppose the graph $G = (V, E)$ models Sydney's road network with roads E and intersections V. The weight of an edge $\overrightarrow{uv}$ records how much time it takes to drive down the road from u to v. Your home is located at intersection s, and you want to identify the quickest journey from your home to any other location in Sydney.

Problem

Instance: a directed graph $G = (V, E)$ with non-negative edge-weights $w(e)$, and a designated source vertex $s \in V$.

We will assume that for every $v \in V$ there is a path from s to v.

Task: find the weight of the shortest path from s to v for every $v \in V$.

Observe that this problem makes the following assumptions:

- There is exactly one source vertex s.
- The graph is directed.
- The edge weights are non-negative.

There are other versions of this problem that make different assumptions, we'll see some of them later in the course.

Note

To find shortest paths from s in an undirected graph, simply replace each undirected edge with two directed edges in opposite directions.

Note

There isn't necessarily a unique shortest path from s to each vertex.

This task is accomplished by a very elegant greedy algorithm developed by Edsger Dijkstra in 1959.

Algorithm Outline

Maintain a set S of vertices for which the shortest path weight has been found, initially empty. S is represented by a boolean array.
For every vertex v, maintain a value d_v which is the weight of the shortest 'known' path from s to v, i.e. the shortest path using only intermediate vertices in S. Initially $d_s = 0$ and $d_v = \infty$ for all other vertices.
At each stage, we greedily add to S the vertex $v \in V \setminus S$ which has the smallest d_v value. Record this value as the length of the shortest path from s to v, and update other d_z values as necessary.

This outline still leaves much work to do. We will examine the following four features of Dijkstra's algorithm:

- **Correctness.** Why is it correct to always add the vertex outside S with the smallest d_v value?
- **Updates.** When v is added to S, for which vertices z must we update d_z, and how do we do these updates?
- **Data structure.** What data structure should we use to represent the d_v values?
- **Time complexity.** What is the time complexity of this algorithm, and how is it impacted by our choice of data structure?

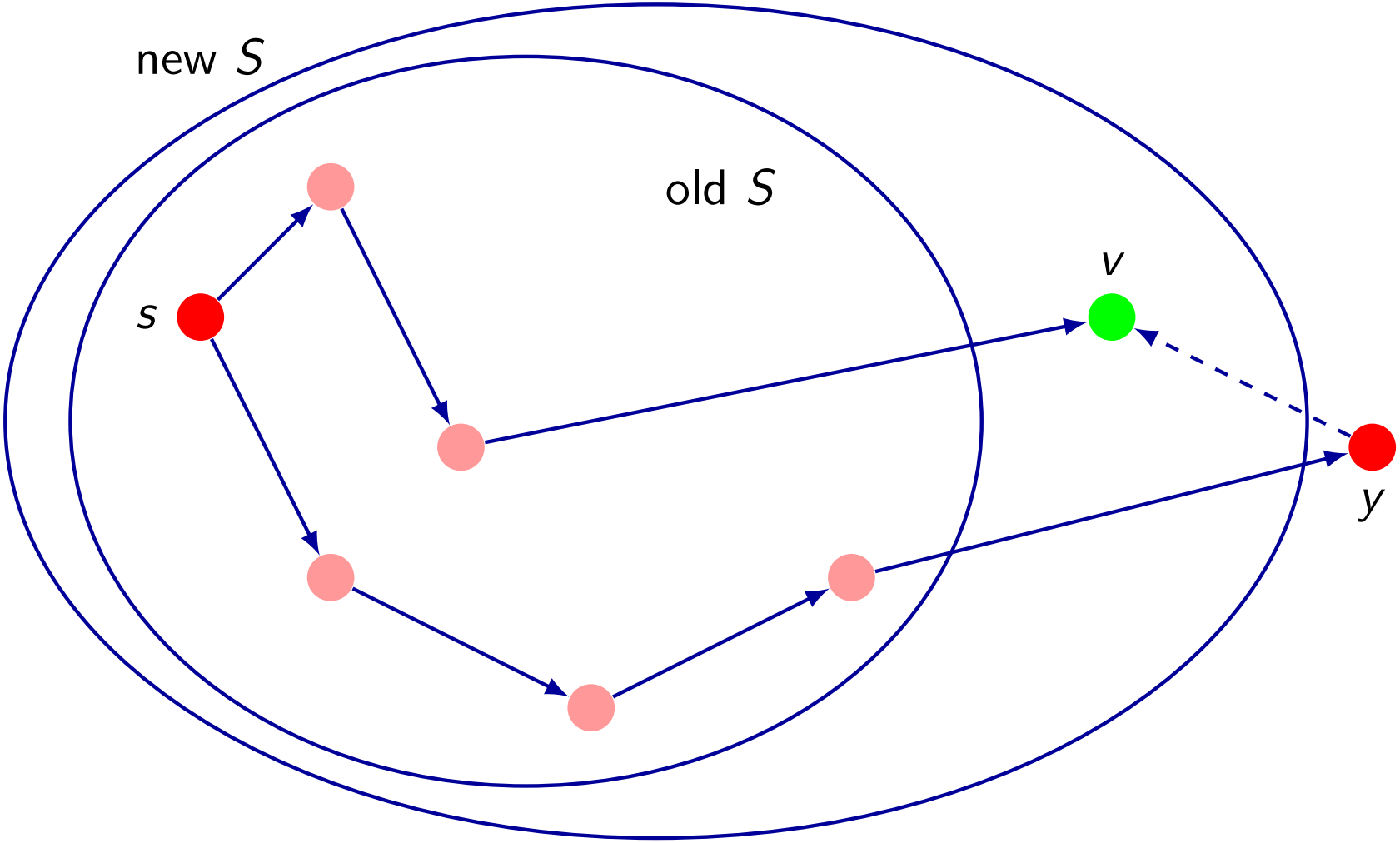
First, we will prove the correctness of Dijkstra's algorithm.

Claim

Suppose v is the next vertex to be added to S. Then d_v is the length of the shortest path from s to v.

Proof

- d_v is the length of the shortest path from s to v using only intermediate vertices in S. Let's call this path p.
- If this were <u>not</u> the shortest path from s to v, there must be some shorter path p' which first leaves S at some vertex y before later reaching v.


new S
old S
s
v
y

Proof (continued)

- Now, the portion of p' up to y is a path from s to y using only intermediate vertices in S.
- Therefore, this portion of p' has weight at least d_y.
- Since all edge weights are non-negative, p' itself has weight at least d_y.

Proof (continued)

- But v was chosen to have smallest d-value among all vertices outside S!
- So we know that $d_v \leq d_y$, and hence the weight of path p is at most that of p'.
- Therefore, d_v is indeed the weight of the shortest path from s to v.

Question

Earlier, we said that when we add a vertex v to S, we may have to update some d_z values. What updates could be required?

Answer

If there is an edge from v to z with weight $w(v, z)$, the shortest known path to z may be improved by taking the shortest path to v followed by this edge. Therefore we check whether

$$d_z > d_v + w(v, z),$$

and if so we update d_z to the value $d_v + w(v, z)$.
As it turns out, these are the <u>only</u> updates we should consider!

Claim

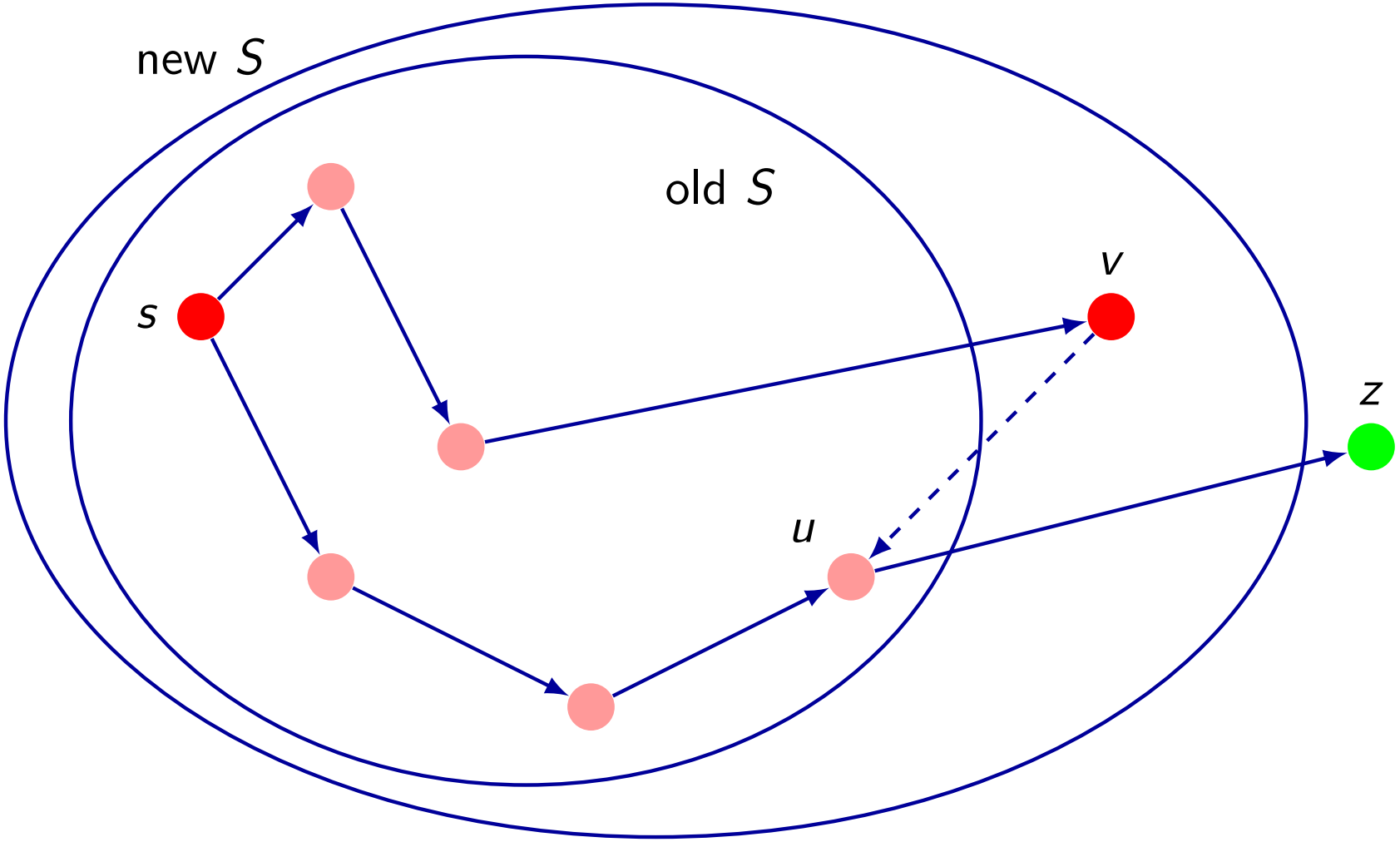
If d_z changes as a result of adding v to S, the new shortest known path to z must have penultimate vertex v, i.e., the last edge must go from v to z.

Proof

- Suppose that adding v to S allows for a new shortest path through S from s to z with penultimate vertex $u \neq v$.
- Such a path must include v, or else it would not be new. Thus the path is of the form

$$p = s \to \cdots \to v \to \cdots \to u \to z.$$

Dijkstra's Algorithm: Updates



Proof (continued)

- Since u was added to S before v was, we know that there is a shortest path p' from s to u which does not pass through v.
- Appending the edge $\overrightarrow{uz}$ to p' produces a path through S from s to z which is no longer than p.
- This path was already a candidate for d_z, so the weight of p is greater than or equal to the existing d_z value.
- This is a contradiction, so the proof is complete.

Dijkstra's Algorithm: Data Structures

- Now, we are ready to consider data structures to maintain the d_v values.
- We need to support two operations:
  - find the vertex $v \in V \setminus S$ with smallest d_v value, and
  - for each of its outgoing edges (v, z), update d_z if necessary.
- We'll start with the simplest data structure: the array.

Let $n = |V|$ and $m = |E|$, and suppose the vertices of the graph are labelled $1, \ldots, n$, where vertex s is the source.

Attempt 1

Store the d_i values in an array $d[1..n]$.
At each stage:

- Perform a linear search of array d, ignoring those vertices already in S, and select the vertex v with smallest $d[v]$ to be added to S.
- For each outgoing edge from vertex v to some $z \in V \setminus S$, update $d[z]$ if necessary.

Question

What is the time complexity of this algorithm?

Answer

- At each of n steps, we perform a linear scan on an array of length n.
- We also run the update procedure (in constant time) at most once for each edge.
- The algorithm therefore runs in $O(n^2 + m)$ time.

Dijkstra's Algorithm: Array

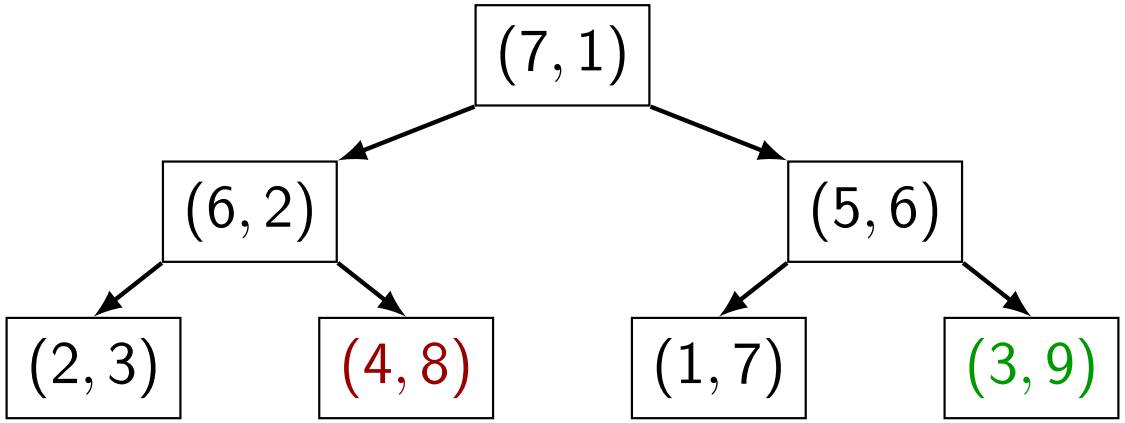
- In a simple directed graph (no self loops or parallel edges) we have $m \leq n(n-1)$, so we can simplify the time complexity expression to just $O(n^2)$.
- If the graph is <u>dense</u> (i.e., $m = \Theta(n^2)$), this linear in the input size.
- Can we do better when $m \ll n^2$?
- For example, road networks are sparse, and have $m = O(n)$.

Dijkstra's Algorithm: Data Structures

- Recall the two operations we need to support:
  - find the vertex $v \in V \setminus S$ with smallest d_v value, and
  - for each of its outgoing edges (v, z), update d_z if necessary.
- So far, we have done the first operation in $O(n)$ using a linear search.
- How can we improve on this?

- The first operation isn't a pure 'find minimum' because we have to skip over vertices already in S.
- Instead, when we add a vertex v to S, we could try deleting d_v from the data structure altogether.
- We now have three operations to support:
  - find minimum,
  - delete minimum, and
  - update any.
- The first two of these suggest the use of a min-heap, but the standard heap doesn't allow us to update arbitrary elements.

- We will use a heap represented by an array $A[1..n]$; the left child of $A[j]$ is stored in $A[2j]$ and the right child in $A[2j+1]$.
- Every element of A is of the form $A[j] = (i, d_i)$ for some vertex i. The min-heap property is maintained with respect to the d-values only.
- We will also maintain another array $P[1..n]$ which stores the <u>position</u> of elements in the heap.
- Whenever $A[j]$ refers to vertex i, we record $P[i] = j$, so that we can look up vertex i using the property $A[P[i]] = (i, d_i)$.

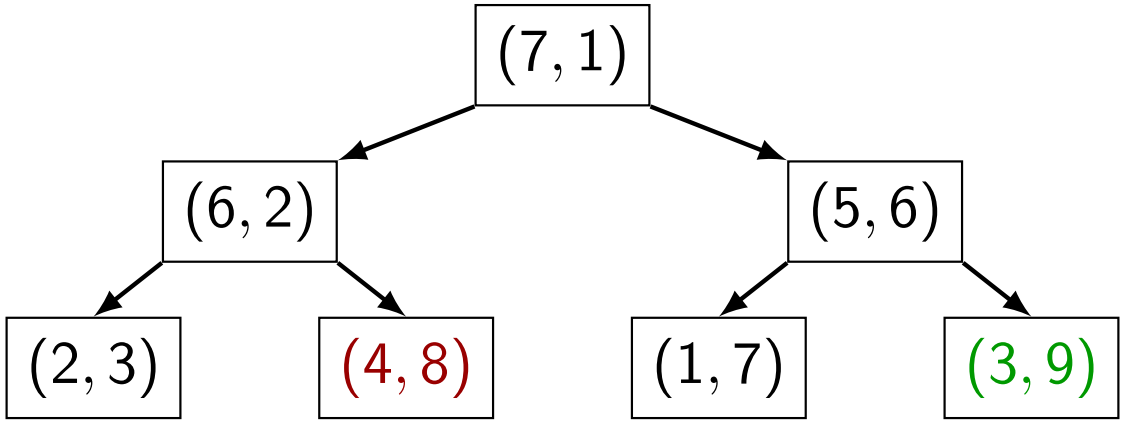


j	1	2	3	4	5	6	7	
$A[j]$	7	6	5	2	4	1	3	i
	1	2	6	3	8	7	9	d_i

i	1	2	3	4	5	6	7
$P[i]$	6	4	7	5	3	2	1

- Updating the d-value of vertex i is now an $O(\log n)$ operation.
  - First, look up vertex i in the position array P. This gives us $P[i]$, the index in A where the pair (i, d_i) is stored.
  - Next, we update d_i by changing the second entry of $A[P[i]]$.
  - Finally, it may be necessary to bubble up or down to restore the min-heap property. In this algorithm, d-values are only ever reduced, so only bubbling up is applicable.
- Accessing the top of the heap still takes $O(1)$, and popping the heap still takes $O(\log n)$.

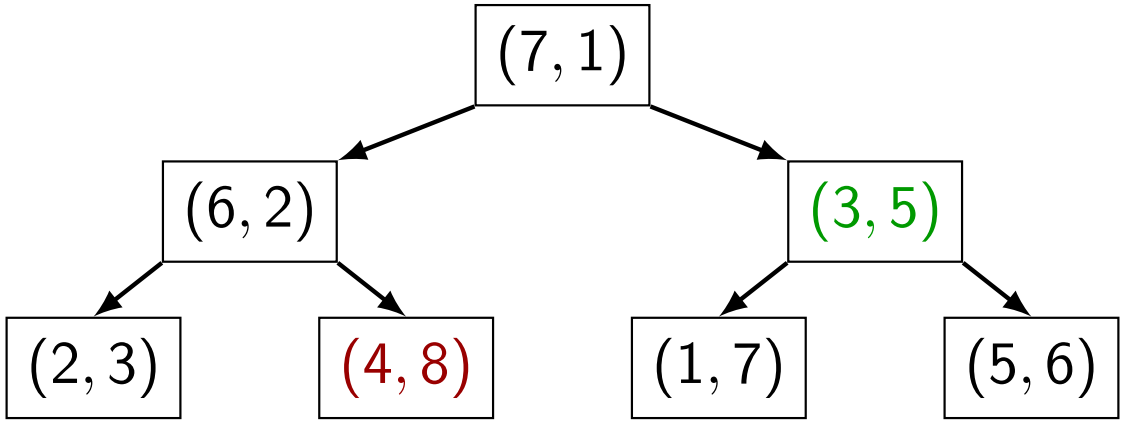
Example. Update d[3] from 9 to 5.
Before the update:



j	1	2	3	4	5	6	7	
$A[j]$	7	6	5	2	4	1	3	i
	1	2	6	3	8	7	9	d_i

i	1	2	3	4	5	6	7
$P[i]$	6	4	7	5	3	2	1

Example. Update d[3] from 9 to 5.
After the update:



j	1	2	3	4	5	6	7	
$A[j]$	7	6	3	2	4	1	5	i
	1	2	5	3	8	7	6	d_i

i	1	2	3	4	5	6	7
$P[i]$	6	4	3	5	7	2	1

Algorithm

Store the d_i values in an augmented heap of size n.
At each stage:

- Access the top of the heap to obtain the vertex v with smallest key and add it to set S.
- Pop the corresponding element d_v from the heap.
- For each outgoing edge from v to some $z \in V \setminus S$, update d_z if necessary.

Question

What is the time complexity of our algorithm?

Answer

- Build the initial heap in $O(n \log n)$ time (faster $O(n)$-time algorithms also exist).
- Each of n stages requires a deletion from the heap (when a vertex is added to S), which takes $O(\log n)$ many steps.
- Each edge causes at most one update of a key in the heap, also taking $O(\log n)$ many steps.
- Thus, in total, the algorithm runs in time $O((n + m) \log n)$.
- But since there is a path from v to every other vertex, we know $m \geq n - 1$, so we can simplify to $O(m \log n)$ total time.

Note

In COMP2521/9024, you may have seen that the time complexity of Dijkstra's algorithm can be improved to $O(m + n \log n)$. This relies on Fibonacci heaps, which has not been taught in this course or any prior course.
Therefore, this improvement will be considered external to our course; you should not use it in assessments unless you also detail the Fibonacci heap construction and operations, and prove the improved time complexity. This will not be the intended solution for any assessable problem in our course.

Note

For sparse graphs, a recent $O(m \log^{2/3} n)$-time algorithm by Duan et al. (2025) is faster than Dijkstra's algorithm.

Table of Contents

1. Motivation
2. Optimal Selection
2.1 Activity selection
2.2 Cell tower placement
3. Detour: graph classes
4. Optimal Ordering
4.1 Tape storage
4.2 Minimising job lateness
5. Optimal Merging
5.1 The Huffman Code
6. Applications to graphs
6.1 Directed graph structure
6.2 Single source shortest paths
7. Matroids
7.1 Minimum spanning trees
8. Puzzle

Definition

A matroid $(S, \mathcal{J})$ consists of a ground set S and a collection $\mathcal{J}$ of subsets of S (called the independent sets) such that:

- **Non-empty:** $\emptyset \in \mathcal{J}$.
- **Hereditary:** If $X \in \mathcal{J}$ and $Y \subseteq X$, then $Y \in \mathcal{J}$.
- **Exchange:** If $X, Y \in \mathcal{J}$ and $|X| > |Y|$, then there exists an $x \in X \setminus Y$ such that $Y \cup \{x\} \in \mathcal{J}$.

- A basis is a maximal independent set, i.e., a basis is an independent set that is not a subset of any other independent set.

- By the exchange property, all bases have the same size.

Matroids: examples

- Let A be an $n \times m$ matrix. The linear matroid of A has ground set $[n] = \{1, \ldots, n\}$. A subset $I \subseteq [n]$ is independent iff the corresponding subset of columns of A is linearly independent.

- Let $G = (V, E)$ be a graph. The cycle matroid of G (a.k.a., the graphic matroid) has ground set E. A subset $F \subseteq E$ is independent iff (V, F) is acyclic.

- Let $G = (V, E)$ be a graph. The matching matroid of G has ground set V. A subset $I \subseteq V$ is independent iff G has a matching that covers I (i.e., there exists a set of edges M that have no endpoint in common such that every vertex in I is incident to an edge in M).

Definition

A weighted matroid $(S, \mathcal{J}, w)$ consists of a matroid $(S, \mathcal{J})$ and a weight function $w : S \to \mathbb{N}_{\geq 0}$.

- The matroid optimisation problem is to compute a basis with maximum total weight of a weighted matroid.

- The independent sets are not given explicitly as input to the algorithm, but we can query whether a given set is independent in time $f(n)$, where $n = |S|$.

Greedy algorithm for matroid optimisation

- Sort the elements of S in decreasing order of weight: $s_1, s_2, \ldots, s_n$ such that $w(s_1) \geq w(s_2) \geq \cdots \geq w(s_n)$.
- Initialise $I \leftarrow \emptyset$.
- For $i = 1$ to n:
  - If $I \cup \{s_i\} \in \mathcal{J}$, then set $I \leftarrow I \cup \{s_i\}$.
- Return I.

Running time

- Sorting the elements takes $O(n \log n)$ time.
- We perform n independence queries, each taking $O(f(n))$ time.
- The total running time is $O(n \log n + nf(n))$.

Let $H = \{h_1, \ldots, h_k\}$ be an optimal basis.
Let $G = \{g_1, \ldots, g_k\}$ be the basis returned by the greedy algorithm. Assume the elements of G and H are indexed in order of decreasing weights.
For the sake of contradiction, suppose that $w(G) < w(H)$.

- Let r be the smallest index such that $w(g_r) < w(h_r)$.
- Consider $G_{r-1} = \{g_1, \ldots, g_{r-1}\}$ and $H_r = \{h_1, \ldots, h_r\}$.
- By the exchange property, there is some $h \in H_r$ such that $G_{r-1} \cup \{h\} \in \mathcal{J}$.
- But $w(h) \geq w(h_r) > w(g_r)$, contradicting the greedy choice of the algorithm.

Theorem

The greedy algorithm returns a maximum-weight basis of a weighted matroid $(S, \mathcal{J}, w)$ *in* $O(n \log n + nf(n))$ *time, where* $n = |S|$ *and any query testing whether a given set is independent can be executed in* $O(f(n))$ *time.*

Exercise

Show that if no two elements of S have the same weight, then there is a unique maximum-weight basis.

- A minimum-weight basis can be computed similarly by sorting the elements in increasing order of weight

- ...or by modifying the weight function: $w'(s) = M - w(s)$ for all $s \in S$, where $M = \max_{s \in S}\{w(s)\}$.

Theorem

A greedy algorithm computes a minimum-weight basis of a weighted matroid $(S, \mathcal{J}, w)$ in $O(n \log n + nf(n))$ time, where $n = |S|$ and any query testing whether a given set is independent can be executed in $O(f(n))$ time.

Table of Contents

1. Motivation
2. Optimal Selection
2.1 Activity selection
2.2 Cell tower placement
3. Detour: graph classes
4. Optimal Ordering
4.1 Tape storage
4.2 Minimising job lateness
5. Optimal Merging
5.1 The Huffman Code
6. Applications to graphs
6.1 Directed graph structure
6.2 Single source shortest paths
7. Matroids
7.1 Minimum spanning trees
8. Puzzle

Definition

$G = (V, E)$ is a connected graph and each edge e in E has a non-negative weight $w(e)$.

- An acyclic subgraph F of G is a subgraph of G that contains no cycle.
- A spanning tree T of G is a connected acyclic subgraph of G on the same vertex set V.
- The weight of a tree T is the sum of all edge weights in T.
- A minimum spanning tree T of G is a spanning tree of G with the smallest weight.

Recall: In the cycle matroid $(E, \mathcal{J})$ of $G = (V, E)$, we have $F \in \mathcal{J}$ iff (V, F) is acyclic.

Proof.

- **Non-empty:** $\emptyset \in \mathcal{J}$ because $(V, \emptyset)$ has no cycle.
- **Hereditary:** If (V, F) has no cycle and $F' \subseteq F$, then (V, F') has no cycle, as the deletion of edges cannot create cycles.
- **Exchange:** Suppose $F_1, F_2 \in \mathcal{J}$ and $|F_1| > |F_2|$. We need to show that F_1 contains an edge e that we can add to F_2 without creating a cycle. We use the following property:

  An acyclic graph (V, F) has $|V| - |F|$ connected components.

  As $|F_1| > |F_2|$, (V, F_1) has fewer connected components than (V, F_2). Select an edge $e \in F_1$ that connects two different components of (V, F_2). Adding e to F_2 does not create a cycle, so $F_2 \cup \{e\} \in \mathcal{J}$.

$\square$

Lemma

The bases of the cycle matroid $(E, \mathcal{J})$ of $G = (V, E)$ are the spanning trees of G.

Proof.

- Assume B is a basis but (V, B) is not connected. As G is connected, there is an edge $e = \{u, v\} \in E$ such that u and v are in different connected components of (V, B). Adding e to B does not create a cycle, so $B \cup \{e\} \in \mathcal{J}$, contradicting the maximality of B. Thus, (V, B) is connected – a contradiction.
- Conversely, consider a spanning tree (V, T) of G. Then $T \in \mathcal{J}$. If T is not maximal, then there is an edge $e \in E \setminus T$ such that $T \cup \{e\} \in \mathcal{J}$. However, adding any edge to a spanning tree creates a cycle – a contradiction.

$\square$

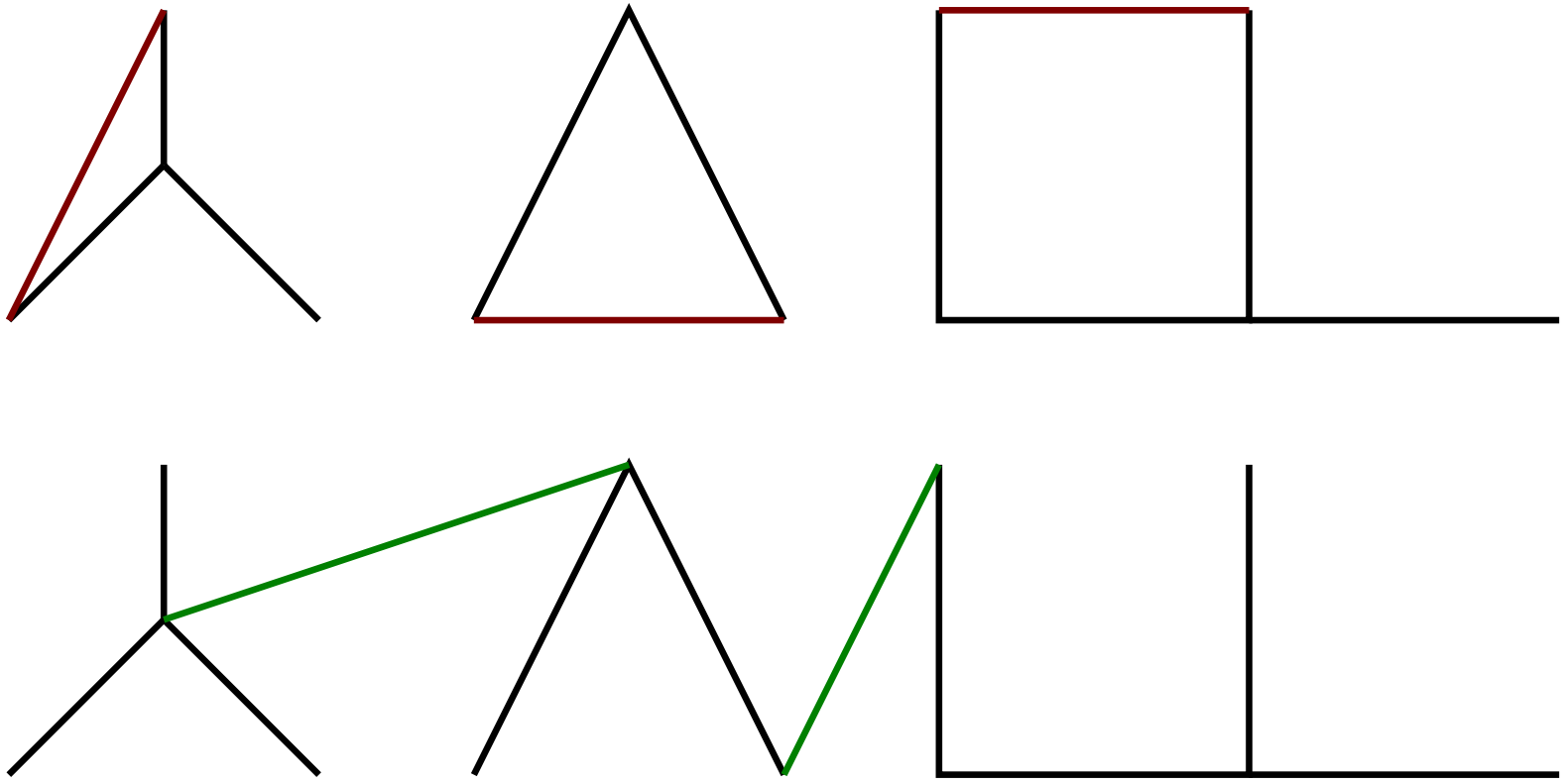
Kruskal's algorithm for Minimum Spanning Tree

- Given a connected graph $G = (V, E)$ and a weight function $w : E \to \mathbb{N}_{\geq 0}$, we want to find a minimum spanning tree of G.
- This is equivalent to computing a basis of the cycle matroid $(E, \mathcal{J})$ of G with minimum total weight.
- This can be done in $O(m \log m + mf(m)) \subseteq O(m \log n + mf(m))$ time using a greedy algorithm for matroid minimisation, where $m = |E|$, if we can test in $f(m)$ time whether a set of edges is acyclic.
- Acyclicity can be tested in linear time, but the union-find data structure allows us to do better.

Kruskal's algorithm

- Edges E are sorted in increasing order by weight.
- An edge e is added if its inclusion does not introduce a cycle in the graph constructed thus far, or discarded otherwise.
- The process terminates when the forest is connected, i.e., when $n - 1$ edges have been added.

- To efficiently implement Kruskal's algorithm, we need to quickly determine whether a certain new edge will introduce a cycle.
- An edge $e = (u, v)$ will introduce a cycle in the forest F if and only if there is already a path between u and v, i.e., u and v are in the same connected component.



We store connected components using the Union-Find data structure. It handles disjoint sets, supporting the following operations:

1. $\textsc{Union}(a, b)$, which changes the data structure by merging the sets containing a and b (say A and B) into a single set $A \cup B$.
2. $\textsc{Find}(a)$, which returns the representative of the set containing a.
   - $\textsc{Find}(a) = \textsc{Find}(b)$ if and only if a and b are in the same set.
   - $\textsc{Find}(a)$ can change over time, as a result of $\textsc{Union}$ operations.

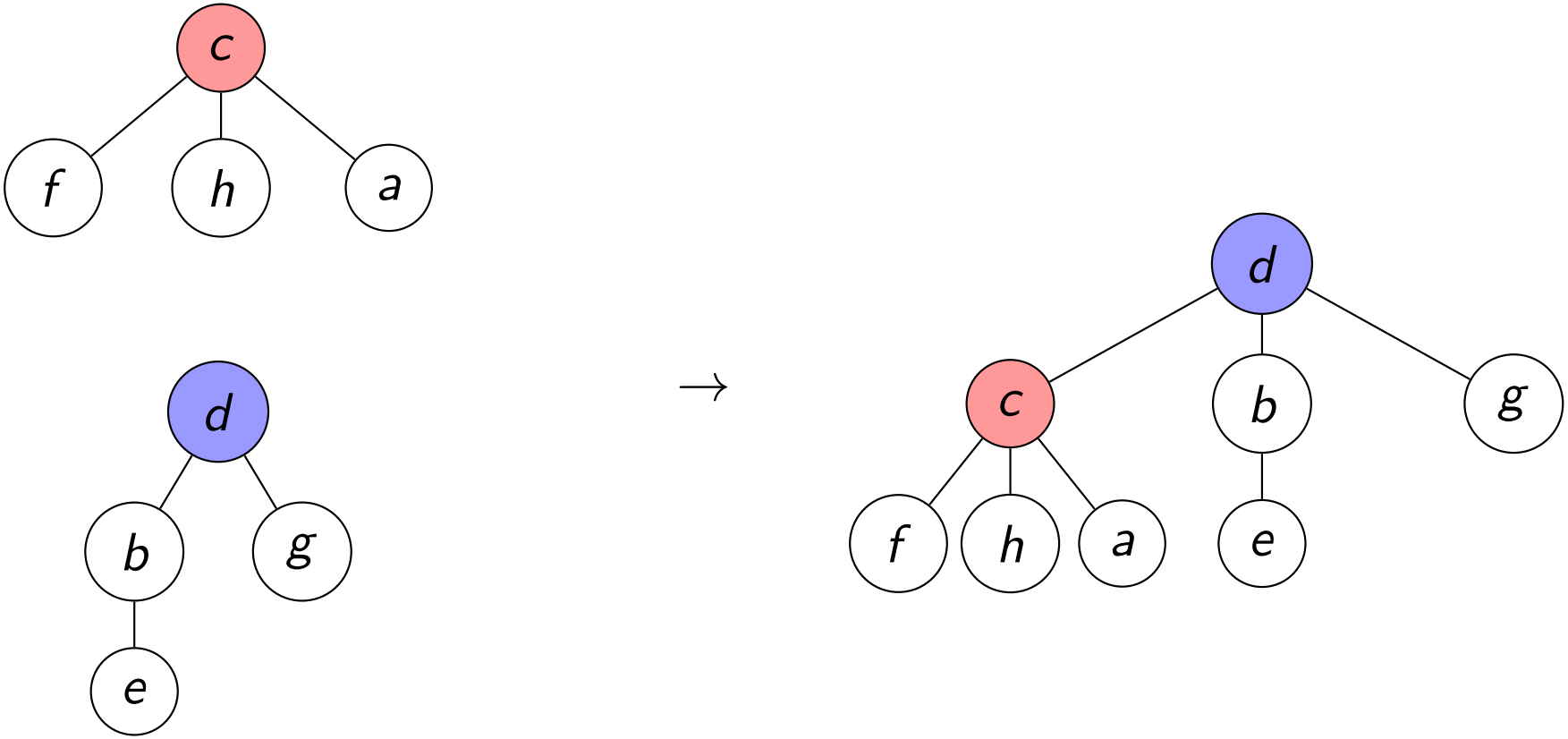
Idea

Maintain a forest, where each tree represents one of our disjoint sets.

- Begin with n vertices and no edges, so initially each vertex belongs to a different set.
- What representative should the FIND operation return?
  - Keep each tree rooted, and return the root.
- How to maintain the rooted structure? Use an array:
  - non-root vertices record their parent, and
  - root vertices record a flag to indicate that they have no parent.

- $\textsc{Find}(a)$ should find the root of the tree containing vertex a.
  - Starting at a, repeatedly traverse the edge to the parent, until there is no parent.
- $\textsc{Union}(a, b)$ should:
  - do nothing if a and b are already in the same tree, or
  - add an edge otherwise, to merge the two trees into one.

We place the edge between <u>the roots of the trees containing</u> a and b.



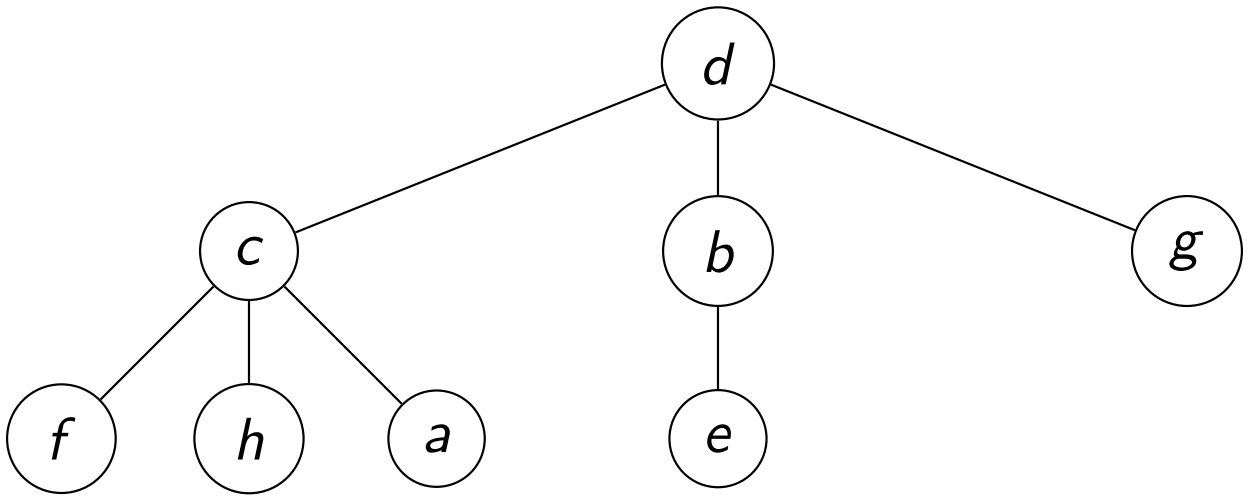
- In the previous example, we made d (the root of b's tree) become the parent of c (the root of a's tree).
- We could have just as easily made c become the parent of d.
- We'll later see how to make this choice more systematically!

In summary, we process $\textsc{Union}(a, b)$ as follows.

1. Run $\textsc{Find}(a)$ and $\textsc{Find}(b)$ to identify the roots of each tree, say c and d respectively.
2. If $c = d$, then do nothing.
3. Otherwise, place an edge between c and d, choosing (for now) arbitrarily which will become the parent of the other.

- A FIND operation takes $O(h)$ time, where h is the height of the tree being traversed.
- A UNION operation involves two FIND operations and potentially one edit to the parent array, so this also takes $O(h)$ time.
- What height can the trees have?
  - If we don't choose wisely when merging two trees together, we may still create a very unbalanced tree, where $h \approx n$.
  - Fortunately, a simple change brings h (and therefore both operations) down to $O(\log n)$.

For each vertex, maintain not only the parent, but also the number of vertices in the subtree rooted at that vertex.



v	a	b	c	d	e	f	g	h
parent(v)	c	d	d	–	b	c	d	c
subtree(v)	1	2	4	8	1	1	1	1

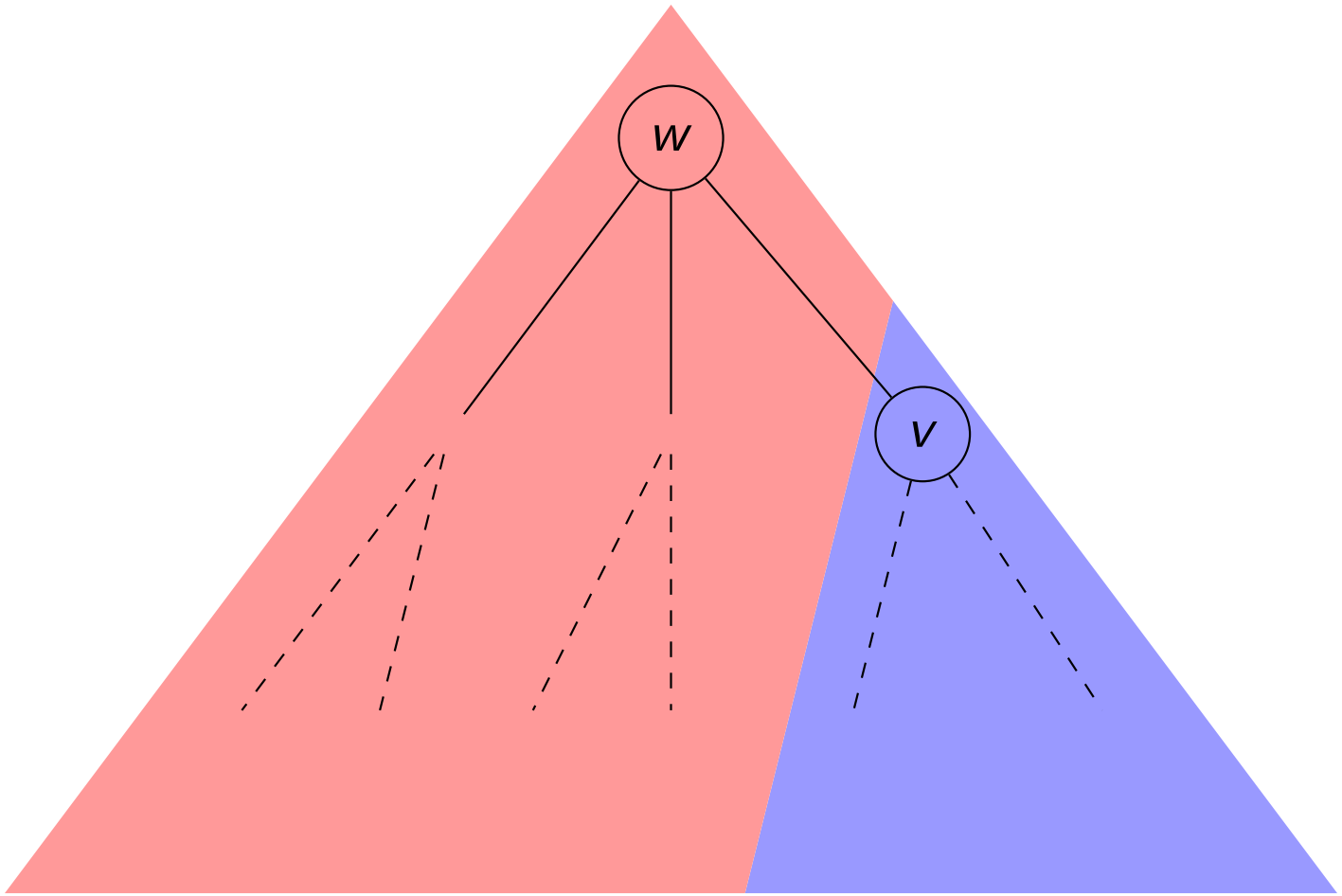
Process $\textsc{Union}(a, b)$ as follows.

1. Run $\textsc{Find}(a)$ and $\textsc{Find}(b)$ to identify the roots of each tree, say c and d respectively.
2. If $c = d$, then do nothing.
3. Otherwise, place an edge between c and d:
   - if $\text{subtree}(c) \geq \text{subtree}(d)$,[1] then set $\text{parent}(d) = c$, or
   - if $\text{subtree}(d) > \text{subtree}(c)$, then set $\text{parent}(c) = d$.

[1] The handling of the equality case is arbitrary; we can set either c or d to be the parent of the other.

Union-Find: Size Heuristic

- What does this have to do with tree height?
- **Key property:** once a vertex v gets a parent w, nothing will be added to v's subtree in the future.
- Just before we placed this edge, subtree(w) was already $\geq$ subtree(v).
- Placing this edge put v's subtree under w, thus increasing subtree(w) by subtree(v).
- The new value of subtree(w) is therefore $\geq 2\,\text{subtree}(v)$.
- As we continue to place edges, subtree(w) might continue increasing, but subtree(v) will not change.


w
v

Claim

The size heuristic guarantees that each tree in the data structure has height at most $\log_2 n$.

Proof

We saw earlier that if w is the parent of v, then $\text{subtree}(w) \geq 2\,\text{subtree}(v)$ <u>at all times</u>.

If the height of the tree is h, there is a path of h parent edges from some leaf vertex v to the root r. This is a chain of h parents.

Proof (continued)

As we traverse that path, the subtree size must at least double at each step, so

$$\begin{aligned} \text{subtree}(v) &= 1 \\ \text{subtree}(\text{parent}(v)) &\geq 2 \\ \text{subtree}(\text{parent}(\text{parent}(v))) &\geq 4 \\ \ldots &\geq \ldots \\ \text{subtree}(r) &\geq 2^h. \end{aligned}$$

However, there are only n vertices in the forest, so $\text{subtree}(r) \leq n$.

It follows that $2^h \leq n$, and therefore $h \leq \log_2 n$.

Union-Find: Size Heuristic

- A FIND operation takes $O(h)$ time, where h is the height of the tree being traversed.
- A UNION operation involves two FIND operations and potentially one edit to the parent array, so this also takes $O(h)$ time.
- The size heuristic guarantees that $h \leq \log_2 n$, so both operations now run in $O(\log n)$.
- This Union-Find data structure is good enough to get the sharpest possible bound on the complexity of Kruskal's algorithm.
- CLRS §19 presents an (equally good) alternative to the size heuristic, known as the rank heuristic, and extends it using path compression to further reduce the amortised complexity of both operations.

Efficient Implementation of Kruskal's Algorithm

- We now use the previously described Union-Find data structure to efficiently perform acyclicity queries.
- The greedy algorithm first sorts m edges of G, which takes time $O(m \log m)$. Since $m < n^2$, we can rewrite this as $O(m \log n^2) \subseteq O(m \log n)$.
- As we progress through the execution of Kruskal's algorithm, we will start with n isolated vertices, which will be merged into connected components until all vertices belong to a single connected component. We use the Union-Find data structure to keep track of the connected components constructed at any stage.

- For each edge $e = (u, v)$ on the sorted list of edges, we use two $\textsc{Find}$ operations to determine whether vertices u and v belong to the same component.

- If they do not belong to the same component, i.e., if $\textsc{Find}(u) = x$ and $\textsc{Find}(v) = y$ where $x \neq y$, we add edge e to the acyclic graph under construction and perform $\textsc{Union}(u, v)$ to place an edge between x and y, merging the trees containing u and v.

- If instead $\textsc{Find}(u) = \textsc{Find}(v)$, there is already a path between u and v, so adding this edge would create a cycle. Therefore, we discard the edge.

Efficient Implementation of Kruskal's Algorithm

- In the worst case, we perform $2m$ FIND operations (checking the endpoints of each edge), each costing $O(\log n)$ time.
- We always perform $n-1$ UNION operations (placing the edges of the MST), each costing $O(\log n)$ time.
- The overall time complexity for the acyclicity checks is therefore $O((m+n)\log n)$.
- As G is connected, $m \geq n-1$, so this can be summarised further to $O(m \log n)$.
- Kruskal's algorithm therefore has $O(m \log n)$ worst-case running time.

- There are two famous greedy algorithms for the minimum spanning tree problem.
- Both algorithms build up a forest, beginning with all n isolated vertices and adding edges one by one.
- Prim's algorithm uses one large component, adding one of the isolated vertices to it at each stage. This algorithm is very similar to Dijkstra's algorithm, but adds the vertex closest to S rather than the one closest to the starting vertex v.
- We will instead focus on Kruskal's algorithm.

Problem

Instance: A complete graph G with weighted edges representing distances between the vertices, and an integer k.

Task: Partition the vertices of G into k disjoint subsets so that the smallest distance between two points belonging to different sets of the partition is as large as possible. Thus, we want a partition into k disjoint sets which are as far apart as possible.

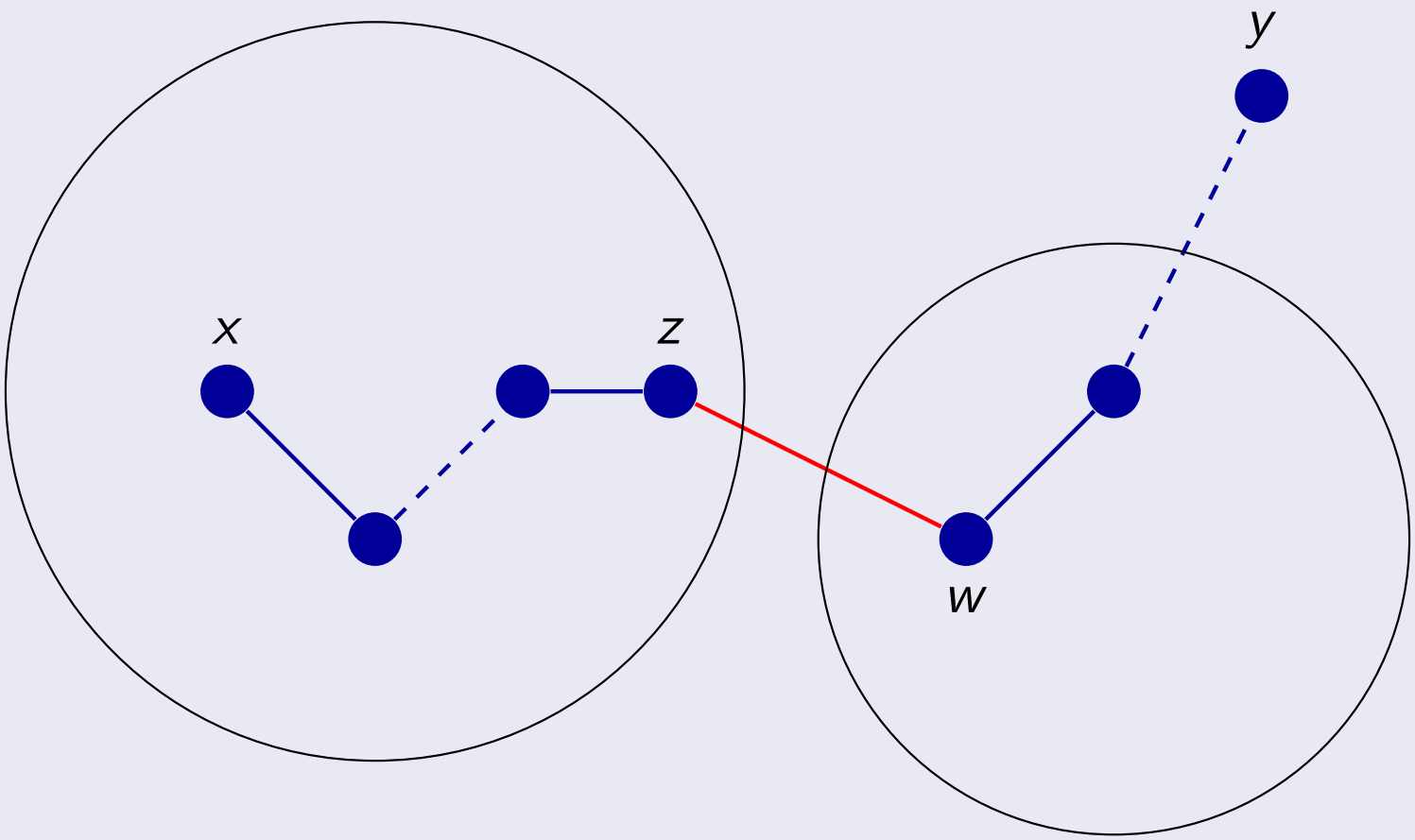
Solution

Sort the edges in increasing order and start performing the usual Kruskal's algorithm for building a minimal spanning tree, but stop when you obtain k connected components, rather than a single spanning tree.

Proof of optimality

- Suppose our clustering achieves a spacing of δ.
  - This means that every edge in the forest has weight at most δ.
- Consider any clustering different from the one produced by our algorithm.
- There must be some pair of vertices x and y that are clustered together in our algorithm but are separated in the other clustering.

Proof of optimality (continued)



Proof of optimality (continued)

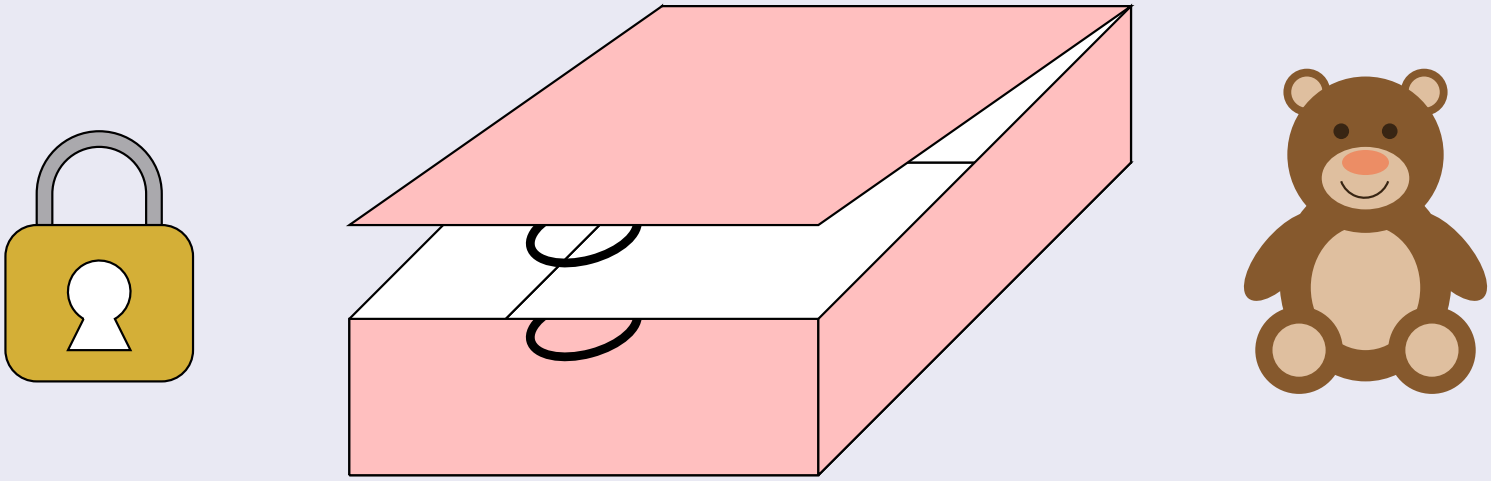
- Since x and y belong to the same tree of our forest, there is a path in that component connecting x and y. Every edge on this path was selected by the partial Kruskal's algorithm, so all these edges must have weight at most δ.
- But in the other clustering (indicated by the circles in the diagram), y is not in the same cluster as x, so this path $x \to y$ must leave x's cluster at some edge (z, w).
- (z, w) is now an edge from one cluster to another, and its weight is at most δ, so the other clustering has spacing at most δ.
- Therefore any other clustering is no better than ours!

- What is the time complexity of this algorithm?
- We have $\Theta(n^2)$ edges; thus sorting them by weight will take $O(n^2 \log n^2)$, which we can simplify to $O(n^2 \log n)$.
- As usual, the running time of Kruskal's algorithm is dominated by sorting the edges.
- So the algorithm has time complexity $O(n^2 \log n)$ in total.

Table of Contents

1. Motivation
2. Optimal Selection
2.1 Activity selection
2.2 Cell tower placement
3. Detour: graph classes
4. Optimal Ordering
4.1 Tape storage
4.2 Minimising job lateness
5. Optimal Merging
5.1 The Huffman Code
6. Applications to graphs
6.1 Directed graph structure
6.2 Single source shortest paths
7. Matroids
7.1 Minimum spanning trees
8. Puzzle

Problem



Bob is visiting Elbonia and wishes to send his teddy bear to Alice, who is staying at a different hotel. Both Bob and Alice have boxes like the one illustrated above, as well as padlocks which can be used to lock the boxes.

Problem (continued)

However, there is a problem. The Elbonian postal service mandates that when a nonempty box is sent, it must be locked. Also, they do not allow keys to be sent, so the key must remain with the sender. Finally, you can send padlocks only if they are locked. How can Bob safely send his teddy bear to Alice?

Hint

The way in which the boxes are locked (via a padlock) is important. It is also crucial that both Bob and Alice have padlocks and boxes. They can also communicate over the phone to agree on the strategy.

There are two possible solutions; one can be called the "AND" solution, the other can be called the "OR" solution. The "AND" solution requires three mailings while the "OR" solution requires only two.

That's All, Folks!!